Transformers for Vision: from Self-Attention to k-means Cross-Attention

Lecture 14

Notes for Probabilistic Models of the Visual Cortex

Alan Yuille

Johns Hopkins University

15 October, 2026

What this lecture covers

Self-attention

Cross-attention and ViT

Axial attention

Mask transformers

Cross-attention as clustering

k-means cross-attention

Summary

Colors used in the figures

One idea runs through the lecture: an attention weight is normalized either over the pixels or over the clusters, and the choice decides what the operation computes. The formulas and the diagrams drawn for these notes mark the two cases with two colors, reserved for them.

- Orange marks a normalization over the pixels (the keys): the weights sum to 1 over the positions p.
- Teal marks a normalization over the clusters (the queries, or the masks): the weights sum to 1 over g.

In the query-key-value diagram, purple marks queries, red keys, blue values and green the input features. Other colors only tell elements apart, and the captions say what they mark; the pictures from the papers keep their own.

A caption that says "TR slide N" took its picture from slide N of the 2025 course slides on transformers (Yuille, TR; numbered as in the .pptx file, whose PDF omits slide 3); "HW slide N" refers to the slides on Axial-DeepLab and MaX-DeepLab by Huiyu Wang, edited by Alan Yuille (Wang, HW).

Transformers were developed for text and adapted to vision

Transformers are the default architecture of large language models (LLMs), and of vision and speech. In language the meaning of a word depends on its context, which adapts to the input and can be very long range, and modeling it is largely why LLMs succeed. In vision the context is more local and less adaptive, and convolutional neural networks (CNNs), with fixed receptive fields, capture it fairly well. Transformers for vision are still work in progress: aspects suited to language are often copied blindly.

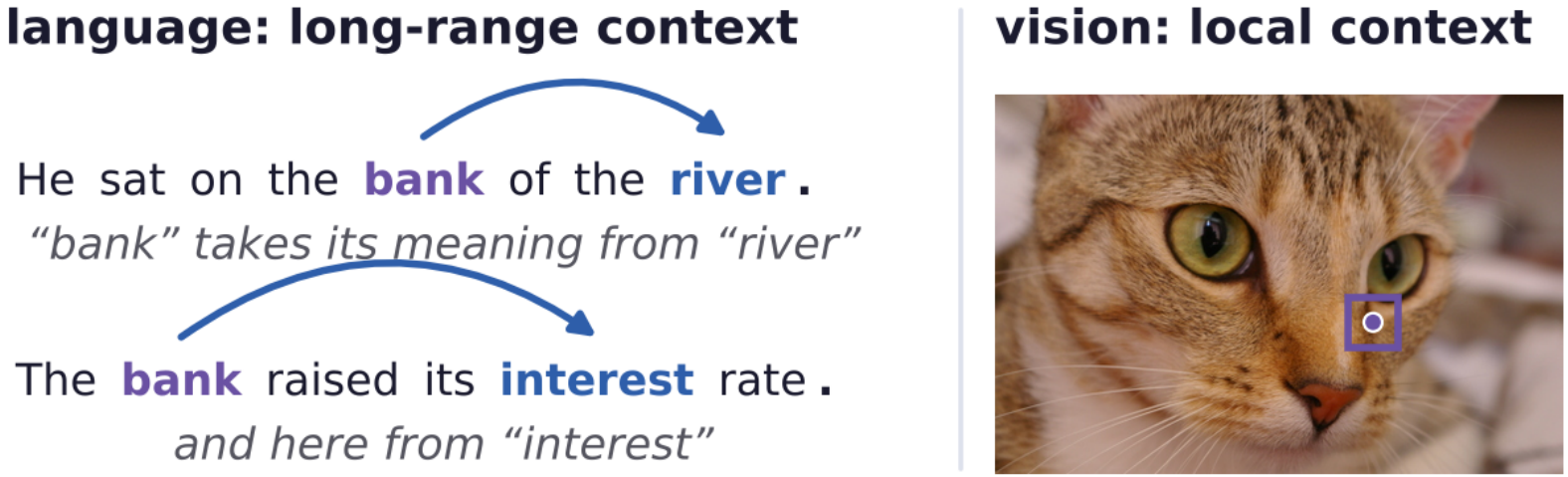


Figure 1: The arcs join "bank" to the distant word that fixes its meaning. In the image (scikit-image "chelsea") the context of a pixel (dot) is its neighborhood (square).

Attention computes a weighted average of feature vectors

A transformer layer takes feature vectors x_p, one at each position p of the input, and outputs new vectors y_o, one at each position o. For an image the inputs can be the colors of patches, the features of a CNN, or the outputs of a previous transformer block. A linear layer computes each output as a weighted sum of the inputs, $y_o = \sum_p w_{op} x_p$. In a convolution the weight depends only on the relative position, $w_{op} = W_{p-o}$; it is non-zero only in a small window around o, and it is fixed once learning has finished: every image is filtered with the same weights. Self-attention lets the weight depend on the inputs themselves:

$$w_{op} = \frac{\exp(x_o^T x_p)}{\sum_{p'} \exp(x_o^T x_{p'})}.$$

The dot product $x_o^T x_p$ is large when the two feature vectors are similar. The exponential and the division by the sum, together called the softmax over p, make the weights positive and make them sum to 1 over p (orange marks this normalization). Hence the key property: the weights are highest between inputs which are similar, wherever they are.

Attention computes a weighted average of feature vectors

The figure computes these weights on the photograph, smoothed a little and cut into 20×30 patches of 15×15 pixels. Each x_p is the mean color of patch p in the Lab color space (lightness and two color-opponent axes), centerd and scaled to unit length, so that $x_o^T x_p$ is the cosine of the angle between two colors; the weights are the softmax over p of $10\, x_o^T x_p$, the factor 10 sharpening them. The query on the fur draws its weight from fur patches all over the cat; the query on the wall from the wall and from the other pale regions.



Figure 2: A convolution at the cyan patch uses a fixed 3×3 window (white). Attention weights of a query patch (cyan) on the fur and on the wall, normalized over all 600 patches and shown scaled to their maximum: bright means a large weight.

Self-attention with query, key and value

In practice each layer separates the roles that a feature vector plays. It learns three matrices W^Q, W^K, W^V and computes, at every position,

$$q_o = W^Q x_o, \qquad k_p = W^K x_p, \qquad v_p = W^V x_p,$$

called the query, the key and the value (terms from natural language processing). The layer outputs

$$y_o = \sum_p \mathrm{softmax}_p(q_o^T k_p)\, v_p, \qquad \mathrm{softmax}_p(q_o^T k_p) = \frac{\exp(q_o^T k_p)}{\sum_{p'} \exp(q_o^T k_{p'})}.$$

The subscript of the softmax names the index it normalizes over. The dot product $q_o^T k_p$ is the affinity between the feature vectors at o and p: the query of o says what o looks for, the key of p what p offers, and the value of p what p passes on. If W^Q and W^K are identity matrices, the affinity is simply the dot product $x_o^T x_p$; with $W^Q \neq W^K$ it need not be symmetric. The update makes the feature vector at o more similar to those at the positions where it has high affinity.

Self-attention with query, key and value

The figure draws the computation for the first of three positions. Each input x_p (green row of circles, above its image patch) is projected to its query, key and value. The query of position 1 meets every key, and each black dot is one affinity $q_1^T k_p$. The gray bar takes the softmax over p; each weight a_{1p} multiplies the value of its position ($\times$ nodes), and the sum gives y_1. Every position repeats this with its own query.

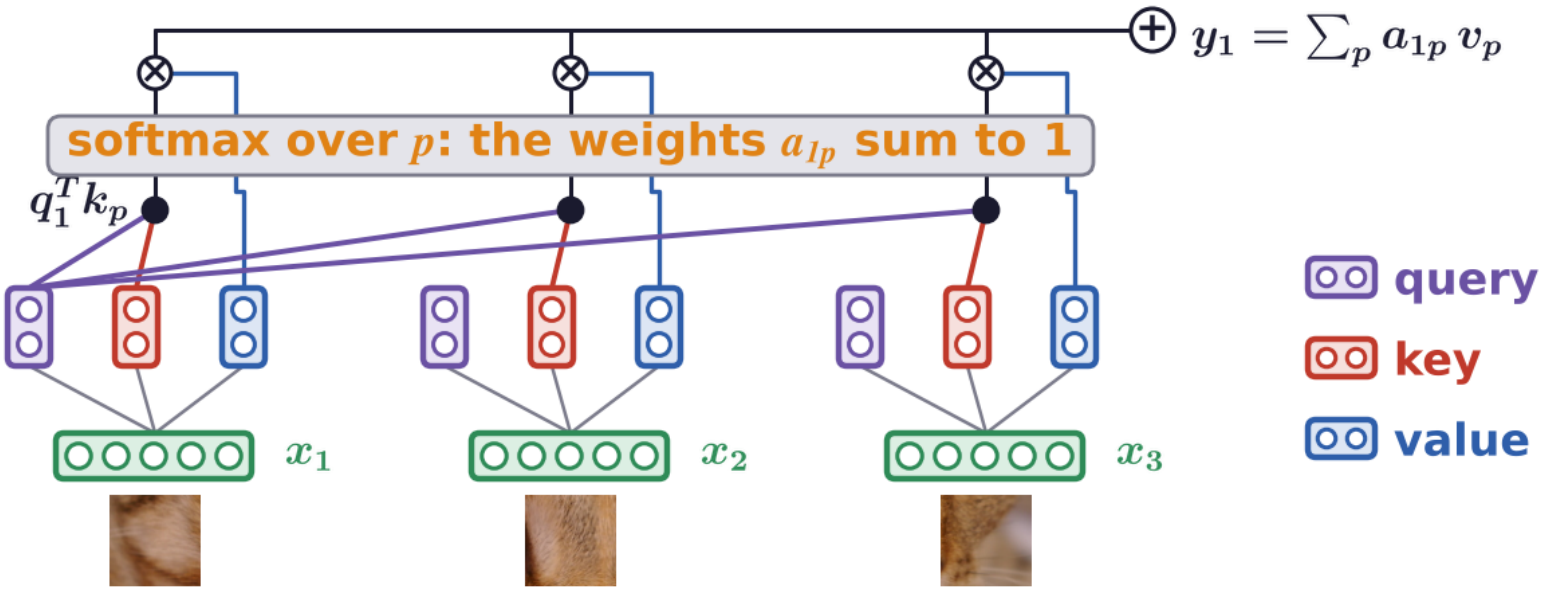


Figure 3: Self-attention for position 1 (after TR slides 10–12, redrawn with three patches of the photograph). Purple: queries; red: keys; blue: values; green: input features.

Scaled dot-product attention is quadratic in the tokens

A token is one input vector x_p: a word, or an image patch. Write d for the dimension of the queries and keys. Vaswani et al. (2017) divide the affinities by $\sqrt{d}$:

$$a_{op} = \frac{\exp(q_o^T k_p/\sqrt{d})}{\sum_{p'} \exp(q_o^T k_{p'}/\sqrt{d})}, \qquad \sum_p a_{op} = 1, \qquad y_o = \sum_p a_{op}\, v_p.$$

If the components of q and k are independent, with mean 0 and variance 1, then $q^T k = \sum_{i=1}^d q_i k_i$ is a sum of d terms of mean 0 and variance 1, so its variance is d. With $d = 64$ the affinities spread over about ± 16, the softmax puts most of its weight on one or a few keys, and its gradients become very small. Dividing by $\sqrt{d}$ brings the variance back to 1.

Every query meets every key. With n tokens there are n^2 affinities, each a dot product of d numbers, so a layer costs $O(n^2 d)$ operations. A vision transformer on the 16×16-pixel patches of a 224×224 image has 196 patch tokens and 38,416 affinities in each attention. Among all the pixels of a 128×256 feature map (a 1024×2048 street image at one eighth resolution) it needs about 1.1×10^9.

Scaled dot-product attention is quadratic in the tokens

The figure shows the matrix of weights for 16 tokens: the photograph cut into a 4×4 grid of patches, numbered row by row. Each token is the mean Lab color of its patch, standardized ($d = 3$), and $W^Q = W^K$ is the identity. Row o holds the weights a_{op} of query o over the 16 keys and sums to 1. Row 10, drawn again on the image, puts its largest weights on token 11 (0.42) and on itself (0.33), the two patches of the most similar color.

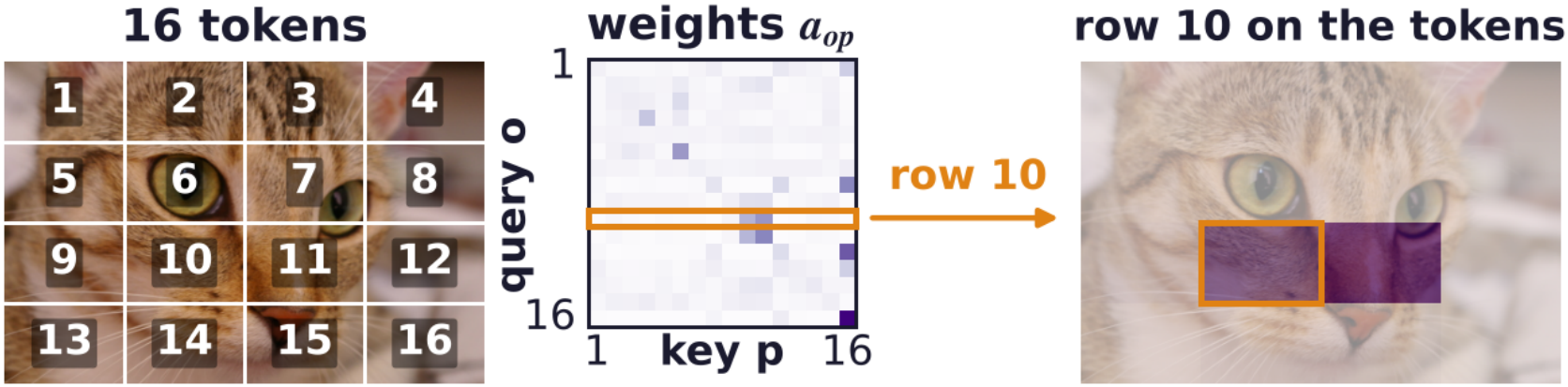


Figure 4: The tokens, the matrix of weights (darker purple: larger weight; rows: queries o, columns: keys p) and row 10 shaded on the tokens (its query outlined in orange).

A transformer is a sequence of transformer blocks

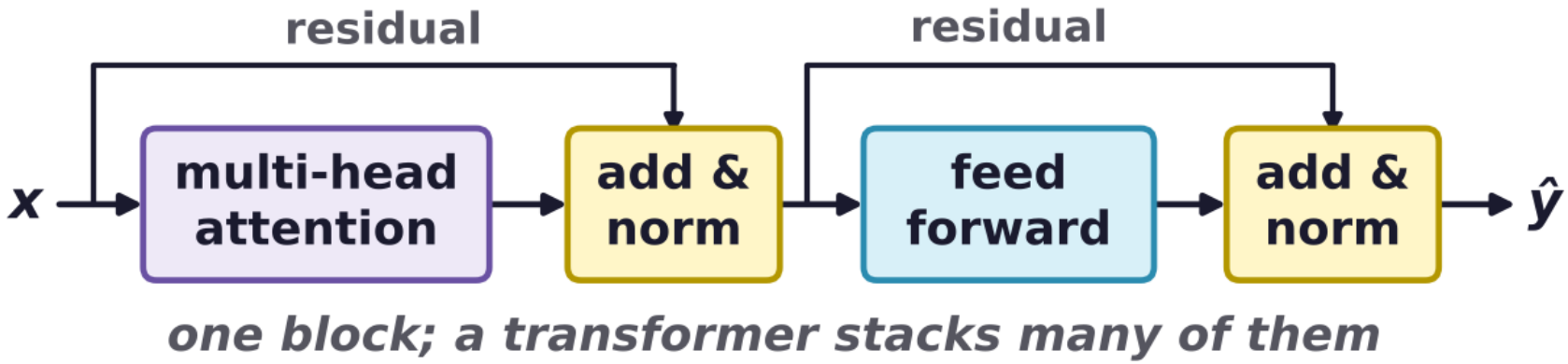
A transformer block has two stages: attention, then a multi-layer perceptron (MLP) f with weights θ applied at each position, each with a residual connection that adds its input to its output:

$$y_o = x_o + \sum_p \text{softmax}_p(q_o^T k_p)\, v_p, \qquad \hat{y}_o = y_o + f(y_o, \theta).$$

The MLP (feed forward in the figures), the same at every position, is a 1×1 convolution: two linear layers with a rectified linear unit (ReLU) between them. Attention is the only interaction between positions. Each residual sum is followed by layer normalization (Ba et al., 2016), which scales each vector to zero mean and unit variance over its components, then applies a learned gain and bias. Multi-head attention runs h attentions in parallel, each with its own smaller matrices W_i^Q, W_i^K, W_i^V, and combines them by a learned W^O:

$$y_o^{(i)} = \sum_p \text{softmax}_p\left(q_o^{(i)T} k_p^{(i)} / \sqrt{d}\right) v_p^{(i)}, \qquad y_o = W^O\left[y_o^{(1)}; \ldots; y_o^{(h)}\right],$$

A transformer is a sequence of transformer blocks



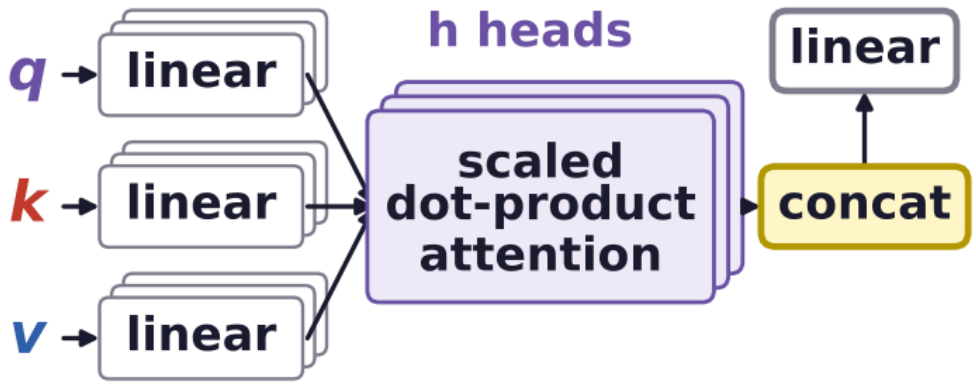


Figure 5: One transformer block and multi-head attention (after Vaswani et al., 2017, Figs. 1–2, TR slides 19–20, redrawn). Arcs: residual connections. In self-attention the inputs q, k, v of the linear layers are all the features x_p; each head projects them to its own queries, keys and values. The base model of Vaswani et al. has 6 encoder and 6 decoder blocks, $h = 8$ heads, vectors of 512 dimensions and $d = 64$ in each head.

Positional encoding gives each feature its position

Attention ignores where its inputs are. Permute the inputs, $x'_p = x_{\pi(p)}$ for a permutation π; the queries, keys and values are permuted in the same way, and

$$y'_o = \sum_p \text{softmax}_p \left(q^T_{\pi(o)} k_{\pi(p)}\right) v_{\pi(p)} = \sum_p \text{softmax}_p \left(q^T_{\pi(o)} k_p\right) v_p = y_{\pi(o)},$$

since the sum runs over the same positions. Permuting the inputs permutes the outputs in the same way, and so does the MLP. In language the order of words is crucial: "The cat sat on the mat" differs from "The mat sat on the cat". In vision, permuting the pixels changes the image.

The remedy puts the position into the feature. The simplest way appends it, $\tilde{x}_p = [x_p; p]$; more often an encoding e_p is added, $\tilde{x}_p = x_p + e_p$. Vaswani et al. use sinusoids, $e_{p,2i} = \sin(p/10000^{2i/D})$ and $e_{p,2i+1} = \cos(p/10000^{2i/D})$ (D: the feature dimension); the vision transformer (ViT) learns e_p. Appending p is crude: relative position matters more than absolute position, and with sinusoids e_{p+k} is a linear function of e_p for every offset k, which may help attention use them.

Positional encoding gives each feature its position

Treating the input as a set has a benefit: the feature vectors need not lie on a regular lattice, and some positions may be missing, as in masked image modeling, where the network predicts patches removed from an image. Positional encoding may also capture spatial relations, such as the nose of a fox lying below its eyes.

In the figure the photograph is cut into 3×3 patches. Self-attention with all the W matrices the identity is run on the nine tokens: queries and keys are the standardized mean Lab colors of the patches, values their mean RGB (red, green, blue) colors, so each output is a color. After a shuffle each patch gets the same output as before, in the shuffled order.

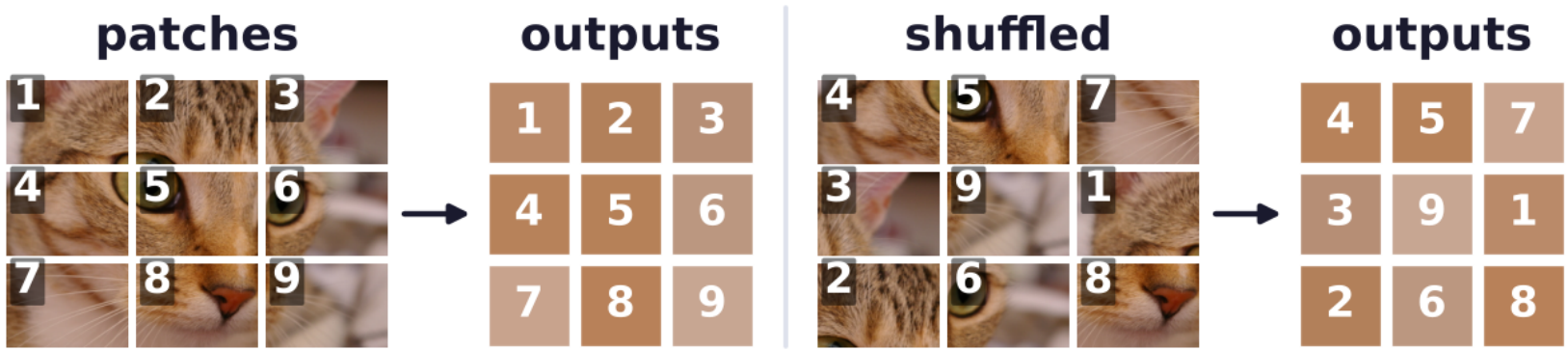


Figure 6: Patches and their self-attention outputs (squares, numbered as their patches), before and after a shuffle of the patches.

The encoder uses self-attention, the decoder cross-attention

The first application of transformers was translation between languages. The encoder converts the input text into features, with self-attention among the input words and an MLP at each word. The decoder converts these features into text, one word at a time; each decoder block has three stages.

- Masked self-attention: each output word attends to the output words produced so far, after a start token. The affinities with later words are set to $-\infty$ before the softmax, so their weights are 0.
- Cross-attention: each output word attends to the encoder's features of the input words. Its queries come from the decoder, its keys and values from the encoder, and its weights sum to 1 over the input words.
- An MLP at each word.

The cross-attention weights specify the correspondence between the words of the input and of the output: when the decoder produces "black" its largest weight falls on "noir". In vision, encoders extract pixel-level features and decoders convert them into object-level representations.

The encoder uses self-attention, the decoder cross-attention

The figure follows one translation, “le chat noir” into “the black cat”. After producing “the black”, the decoder receives “start the black” as its input and predicts “the black cat”: each position predicts the word that follows it.

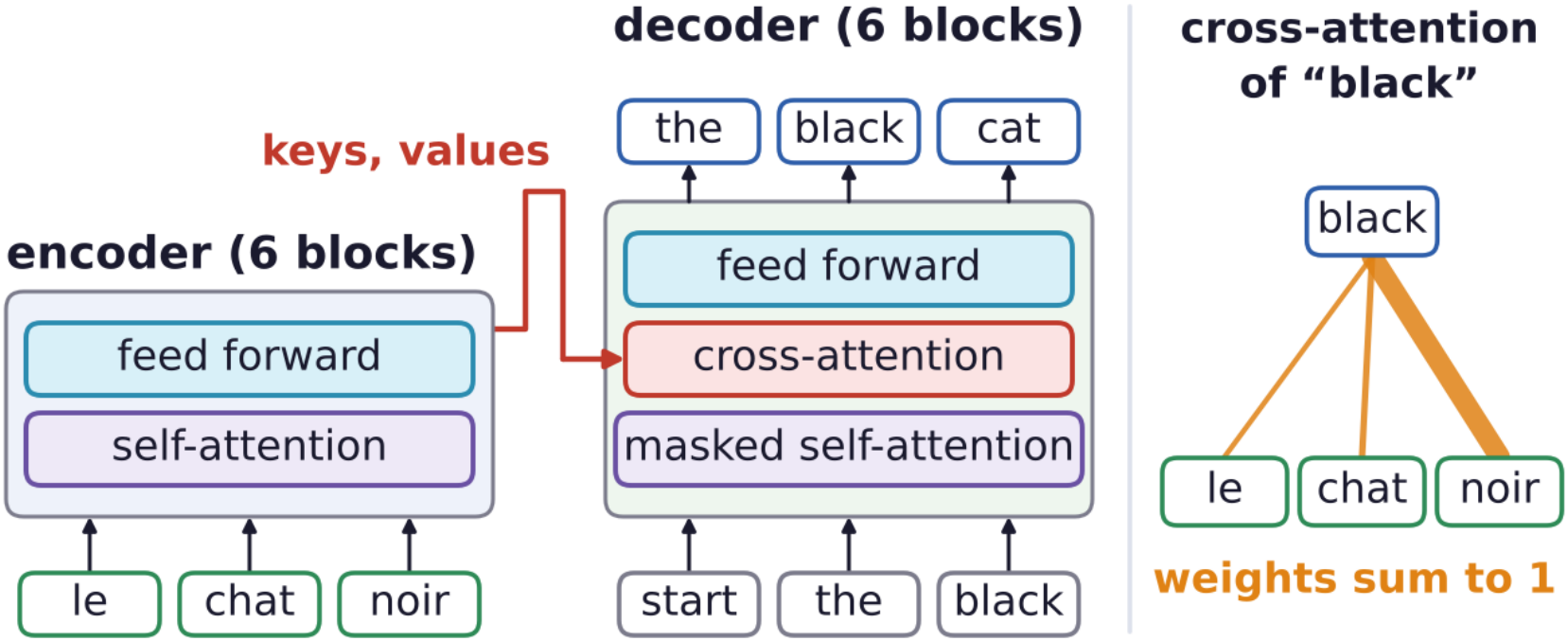


Figure 7: An encoder-decoder transformer (after Vaswani et al., 2017, Fig. 1, TR slides 21–27, redrawn). The encoder’s output feeds the decoder’s cross-attention as keys and values (red). The last panel: cross-attention weights of the output word “black” over the input words (line width; illustrative values).

Cross-attention: queries from one set, keys from another

Cross-attention generalizes self-attention: the queries come from one set of vectors, the keys and values from another. The most useful case introduces N latent variables c_g, vectors that are learned or updated, which query the input features: with $q_g = W^Q c_g$, $k_p = W^K x_p$ and $v_p = W^V x_p$,

$$c_g \leftarrow c_g + \sum_p \frac{\exp(q_g^T k_p)}{\sum_{p'} \exp(q_g^T k_{p'})} v_p .$$

The weights sum to 1 over the inputs p (orange): each latent variable looks for its match in the image. The latent variables can serve as outputs, for example one per object, and can also be updated together with the image features in two streams.

Stereo gives a case with pixel queries. Two cameras side by side see a point of the scene in the same row of both images, shifted horizontally by its disparity, which is larger for nearer points. The context-enhanced stereo transformer (CSTR; Guo et al., 2022) estimates it by cross-attention: a pixel o of the left image queries the pixels p of the same row of the right image, and the weights a_{op} peak at the match.

Cross-attention: queries from one set, keys from another

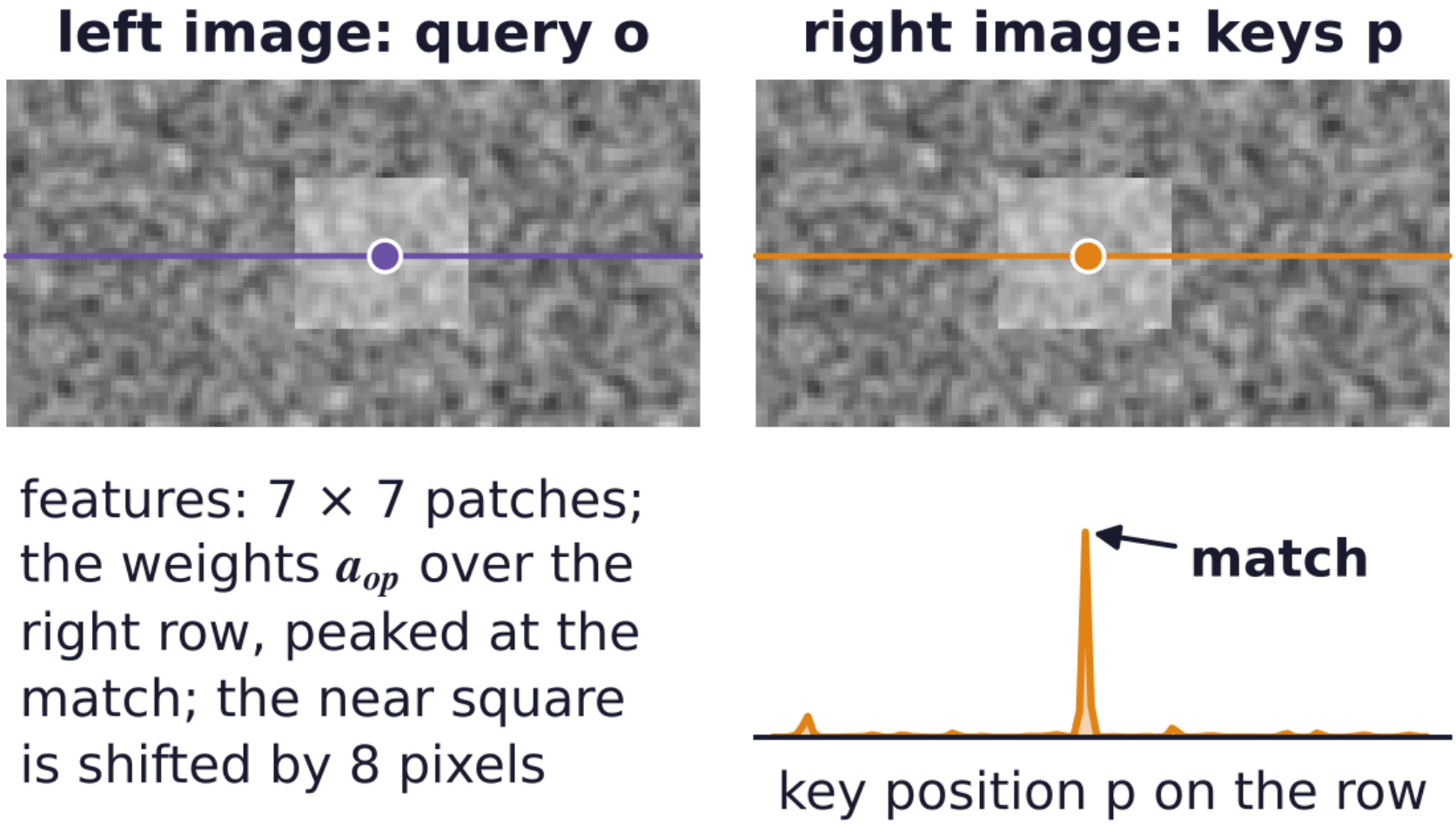


Figure 8: Stereo on a synthetic pair: a nearer square is 8 pixels further left in the right image. Features: normalized 7×7 patches. Pixel 65 of the left row (purple dot) queries the right row; its weights (orange, softmax of 6 times the correlation) peak at pixel 57 (orange dot).

ViT: the pioneer of transformer architectures for vision

The vision transformer (ViT; Dosovitskiy et al., 2021) divides the image into 16×16-pixel patches, 196 for a 224×224 image, treated as the words of a sentence. Each patch is flattened into $16 \times 16 \times 3 = 768$ numbers and mapped by a learned linear projection to a token; learned position embeddings are added, and a learned class token is prepended. An encoder only follows: ViT-Base has 12 blocks of self-attention and MLP, tokens of 768 dimensions and 12 heads (86 million parameters). An MLP head reads the class from the output at the class token.

Trained on ImageNet alone (1.3 million images, 1,000 classes) without strong regularization, ViTs are a few percentage points less accurate than ResNets (residual networks; He et al., 2016) of comparable size. Pre-trained on JFT-300M, an internal Google dataset of 303 million images in 18,000 classes, and fine-tuned on ImageNet, the large ViTs outperform the large ResNets: large-scale training trumps the inductive bias (locality, translation invariance) built into convolution. Strengths: long-range interaction through global attention, and performance that scales with the data. Weaknesses: the quadratic cost, the lack of locality, and the coarse 16×16 patches, which lose low-level detail.

ViT: the pioneer of transformer architectures for vision

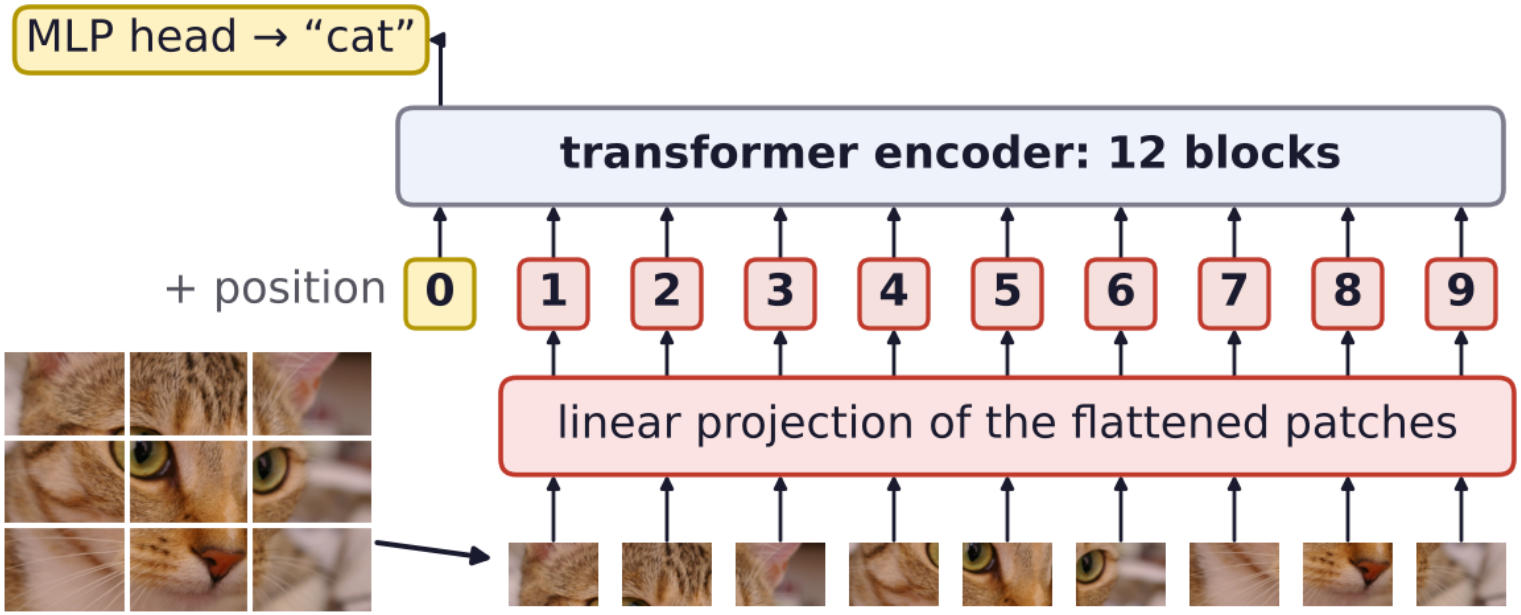


Figure 9: The vision transformer (after Dosovitskiy et al., 2021, Fig. 1, TR slide 28, redrawn). The image is cut into patches (3×3 drawn; a 224×224 image has 14×14); each flattened patch is projected linearly to a token (1–9), the class token (0, yellow) is prepended, and position embeddings are added. The encoder's output at the class token goes to the MLP head.

Visual signals differ from text: scale, locality, position

Three properties of images have no counterpart in text (an example adapted from Ross Girshick and Ze Liu). Multi-scale: a cat can appear at any size, while "I am a fat fat cat cat" is not a valid sentence. Locality: neighboring pixels are similar (the grass), while neighboring words need not be ("I like the green grass"). Translation invariance: a cat is a cat anywhere in the image, while moving words changes a sentence ("Fat cat is me").

Two families of models build these properties back in. The Swin transformer (shifted windows; Liu et al., 2021) computes self-attention inside windows of 7×7 patches, so its cost grows linearly with the image size. Consecutive blocks shift the windows by 3 patches, so that information crosses the window borders, and patch merging between stages builds feature maps at 1/4, 1/8, 1/16 and 1/32 of the image resolution, a hierarchy like that of a CNN. Hybrid models such as MOAT (alternating mobile convolution and attention; Yang et al., 2023), TransUNet and ViTamin combine convolutions with attention: the convolutions supply locality, translation invariance and an efficient spatial hierarchy, and the attention supplies global interaction.

Visual signals differ from text: scale, locality, position



Figure 10: The three properties on one photograph (TR slide 33, after Ross Girshick and Ze Liu; rebuilt from its two pictures). Yellow boxes: the cat pasted at five sizes, two patches of grass, the cat at two positions. The sentences show the contrasting case in text.

Panoptic segmentation gives every pixel a class and instance

Semantic segmentation gives every pixel a class. Instance segmentation gives each countable object, or thing (car, person), its own mask. Panoptic segmentation (Kirillov et al., 2019) unifies the two: every pixel gets a class, and every pixel of a thing also gets an instance; the amorphous regions, the stuff (road, sky), get a class only.

Panoptic quality (PQ) scores recognition and segmentation together. A predicted and a true segment of the same class are matched when their intersection over union (IoU: the area of the overlap divided by the area of the union) exceeds 0.5; since the segments of an image do not overlap, the matching is unique. With TP the matched pairs $(s, \hat{s})$, FP the unmatched predictions and FN the unmatched true segments,

$$\mathrm{PQ} = \underbrace{\frac{\sum_{\mathrm{TP}} \mathrm{IoU}(s, \hat{s})}{|\mathrm{TP}|}}_{\text{segmentation quality}} \times \underbrace{\frac{|\mathrm{TP}|}{|\mathrm{TP}| + \frac{1}{2}|\mathrm{FP}| + \frac{1}{2}|\mathrm{FN}|}}_{\text{recognition quality}},$$

computed for each class and averaged over the classes; PQ Th and PQ St average over the thing and the stuff classes only.

Panoptic segmentation gives every pixel a class and instance

Four papers by Huiyu Wang, Qihang Yu and colleagues, with Alan Yuille and Liang-Chieh Chen, develop transformers for this task within DeepLab, a family of segmentation networks: Axial-DeepLab, MaX-DeepLab (mask transformer), CMT-DeepLab (clustering mask transformer) and kMaX-DeepLab (k-means mask transformer). Their results come from two datasets: COCO (Common Objects in Context; Lin et al., 2014), whose panoptic labels have 80 thing and 53 stuff classes, and Cityscapes (Cordts et al., 2016), street scenes filmed from a car, with 19 classes, 8 of them things.

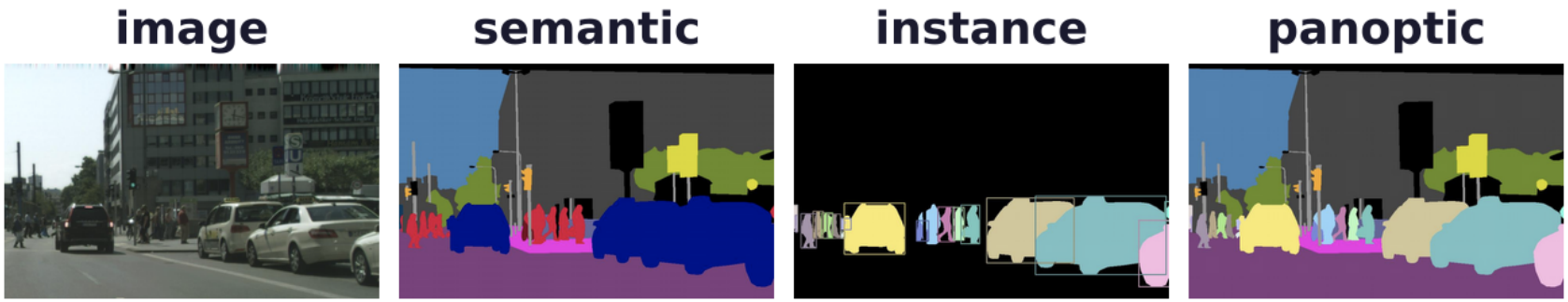


Figure 11: The three tasks on a Cityscapes image (HW slide 31, from Kirillov et al., 2019, Fig. 1): the image, semantic segmentation (one color per class), instance segmentation (a mask and a box per thing, on black) and panoptic segmentation (both).

Self-attention can be global or restricted to a local square

A layer is stand-alone when it alone can build the network, as convolution does (Ramachandran et al., 2019). On an $H \times W$ feature map (W a width), with $\mathcal{N}_{m \times m}(o)$ the $m \times m$ square around o and $\mathcal{N}$ the map:

$$\text{convolution:} \quad y_o = \sum_{p \in \mathcal{N}_{\boldsymbol{m} \times \boldsymbol{m}}(o)} W_{p-o}\, x_p$$

$$\text{global self-attention:} \quad y_o = \sum_{p \in \mathcal{N}} \text{softmax}_p(q_o^T k_p)\, v_p$$

$$\text{local self-attention:} \quad y_o = \sum_{p \in \mathcal{N}_{\boldsymbol{m} \times \boldsymbol{m}}(o)} \text{softmax}_p(q_o^T k_p)\, v_p$$

method	stand-alone	long range	cost
convolution	✓	×	$O(HWm^2)$
global (non-local) self-attention	×	✓	$O(H^2W^2)$
local self-attention	✓	×	$O(HWm^2)$

Global (non-local; Wang et al., 2018) attention is too costly to stand alone; local attention is not long range.

Axial attention factorizes 2D attention into height and width

Axial attention (Ho et al., 2019; criss-cross attention, Huang et al., 2019, is a relative) attends along one axis at a time. A width-axis layer attends along the row through o,

$$y_o = \sum_{p \in \mathcal{N}_{\mathbf{1} \times \boldsymbol{m}}(o)} \text{softmax}_p(q_o^T k_p)\, v_p,$$

where $\mathcal{N}_{1 \times m}(o)$ is the segment of m pixels of that row centerd at o; a height-axis layer does the same along the column. Axial-DeepLab (Wang et al., 2020) sets the span m to the whole row or column in its main models and applies a height-axis layer, then a width-axis layer.

Axial attention factorizes 2D attention into height and width

Two layers reach the whole image. Write a position as (row i, column j). After the height-axis layer, the feature at (i, j') carries information from the whole column j'; the width-axis layer then lets (i, j) gather the features (i, j') of every column j'. Every pixel reaches every other in two steps, at a cost of $O(HWm)$ per layer: stand-alone and long range. Global attention would cost $O(H^2W^2)$.

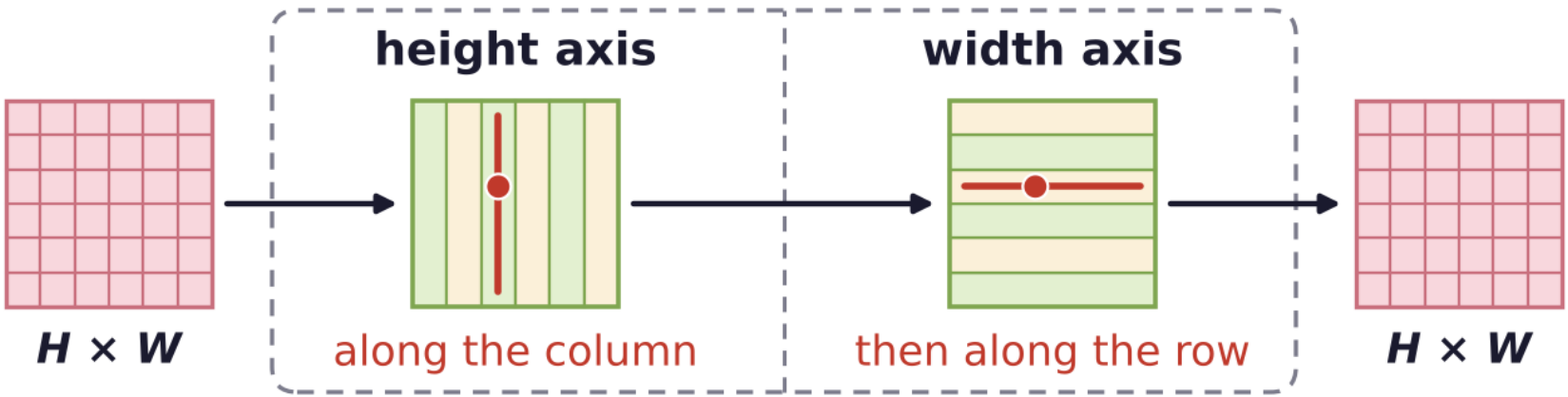


Figure 12: Axial attention (after HW slides 7–8, redrawn): the neighborhoods (red lines) of one pixel (red dot), along its column and then along its row.

Position-sensitive attention adds three positional terms

Self-attention ignores positions, while convolution is position-aware through W_{p-o}. Stand-alone self-attention adds a query-dependent bias $q_o^T r^q_{p-o}$, where r^q_{p-o} is a learned vector for the relative offset $p-o$ (relative position representations were introduced for text by Shaw et al., 2018). Axial-DeepLab adds two more terms:

$$y_o = \sum_{p \in \mathcal{N}_{\mathbf{1}\times \boldsymbol{m}}(o)} \text{softmax}_p \left(q_o^T k_p + \underbrace{q_o^T r^q_{p-o}}_{\text{query bias}} + \underbrace{k_p^T r^k_{p-o}}_{\text{key bias}} \right) \left(v_p + \underbrace{r^v_{p-o}}_{\text{position retrieval}} \right).$$

With the query bias, the query decides which offsets to attend to. The key k_p could also carry information about which locations should attend to it, hence the key bias. Position retrieval lets the output also retrieve relative positions. r^q, r^k, r^v are shared by the heads of a layer.

method	stand-alone	long range	position
convolution	✓	×	✓
global (non-local) self-attention	×	✓	×
local (stand-alone) self-attention	✓	×	✓
Axial-DeepLab	✓	✓	✓✓✓

The axial block replaces the 3×3 convolution of a ResNet

The residual bottleneck block is the unit of a ResNet: a 1×1 convolution reduces the channels, a 3×3 convolution mixes neighboring positions, a 1×1 convolution restores the channels, and the input is added to the output. The axial block replaces the 3×3 convolution with a height-axis and a width-axis attention layer, each with 8 heads. Making every block of a ResNet-50 axial, the stem (its first layers) included, halves the number of parameters.

The table gives ImageNet classification. Params: the number of parameters; M-Adds: multiply-adds per image; top-1: the percentage of images whose most probable class is correct.

ImageNet	params	M-Adds	top-1 (%)
ResNet-50	25.6M	4.1B	76.9
full attention (stand-alone)	18.0M	3.6B	77.6
full axial attention	**12.5M**	**3.3B**	**78.1**

Full axial attention is the most accurate of the three, with the fewest parameters and operations.

The axial block replaces the 3×3 convolution of a ResNet

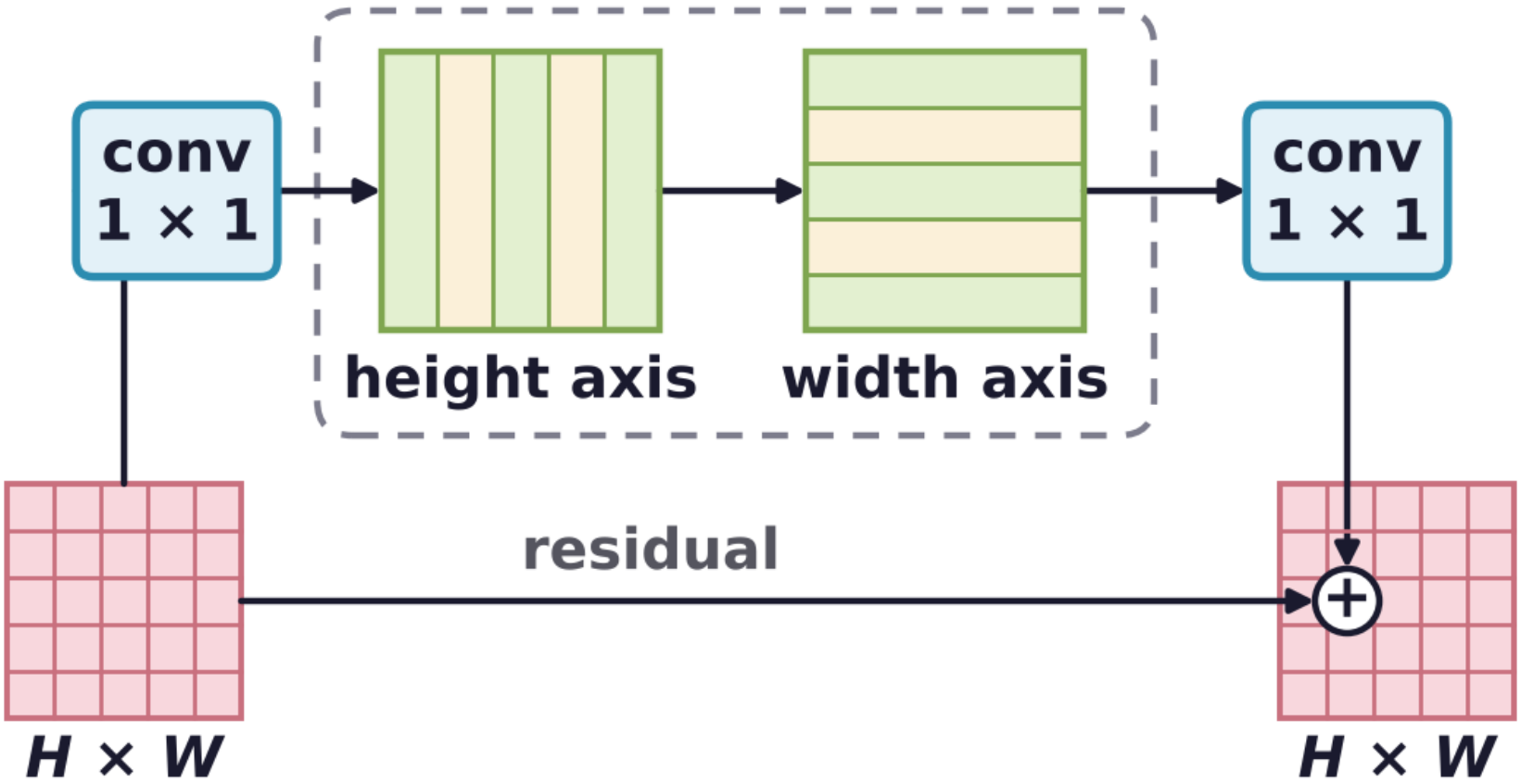


Figure 13: The axial block (after HW slide 19, redrawn): a 1×1 convolution, height-axis attention, width-axis attention (inside the dashed box), a 1×1 convolution, and the residual connection from the input to the sum ($+$).

Long-range axial attention helps panoptic segmentation

On the Cityscapes validation images (val), Axial-DeepLab-S (small) has a higher PQ, AP and mIoU than ResNet-50 and than local attention, with fewer parameters and operations. AP is the average precision of the instance masks (the area under the precision-recall curve, averaged over IoU thresholds); mIoU is the IoU of each semantic class, averaged over the classes. The first three rows use atrous spatial pyramid pooling (ASPP), a head of parallel dilated convolutions.

Cityscapes val	params	M-Adds	PQ	AP	mIoU
ResNet-50, ASPP	30.0M	390.0B	59.8	32.6	77.8
local attention, ASPP	22.5M	332.9B	60.9	30.0	78.2
position-sensitive, ASPP	22.5M	341.9B	61.5	33.1	79.1
Axial-DeepLab-S	**12.1M**	**220.8B**	62.6	34.9	80.5
Axial-DeepLab-L	44.9M	687.4B	**63.9**	**35.8**	**81.0**

Position-sensitive attention gains over local attention (61.5 against 60.9 PQ at nearly the same cost), and the axial models gain again.

Long-range axial attention helps panoptic segmentation

With span m each axial layer attends to m pixels and the two layers cover an $m \times m$ square. The span of the large Axial-ResNet grows from 5×5 to 65×65 (Cityscapes val, no ASPP). PQ and mIoU rise at every step; AP dips from 31.3 to 31.1 at 9×9 and then rises to 36.3, at a marginal extra cost.

model (span m)	M-Adds	PQ	AP	mIoU
ResNet-101	530.0B	59.9	31.9	74.6
5×5	617.4B	59.1	31.3	74.5
9×9	622.1B	61.2	31.1	77.6
17×17	631.5B	62.8	34.0	79.5
33×33	650.2B	63.8	35.9	80.2
65×65	687.4B	**64.2**	**36.3**	**80.6**

The 65×65 row and the Axial-DeepLab-L row have the same size but differ by a few tenths; the paper does not explain the difference. Huiyu Wang's summary of the impact: axial attention was tested in the ViT paper (Axial-ResNet and Axial-ViT, in its appendix), and AlphaFold 2 (Jumper et al., 2021) uses a variant of axial attention for protein structure.

Earlier methods split panoptic segmentation into sub-tasks

Before MaX-DeepLab, panoptic segmentation was solved through a tree of surrogate sub-tasks: stand-in problems whose solutions are combined into the answer. Panoptic-FPN (feature pyramid network; Kirillov, Girshick, He and Dollár, 2019) merges a semantic segmentation with an instance segmentation. The instance segmentation is decomposed into box detection and box-based segmentation, and the box detection into the classification and the regression of anchors, boxes of preset sizes and shapes placed all over the image. Every level brings hand-designed modules: anchors, rules for assigning boxes, non-maximum suppression (which keeps the most confident of overlapping detections) and rules for merging.

Each sub-task can fail on its own. Axial-DeepLab's panoptic head regresses, at each pixel, the offset to the center of its object; when two centers are close, as for a dog sitting on a chair, the pixels of the two objects are grouped together. DetectoRS, a box-based detector, classifies boxes instead of masks; the chair's box has a low confidence, so the chair's mask is filtered out.

Earlier methods split panoptic segmentation into sub-tasks

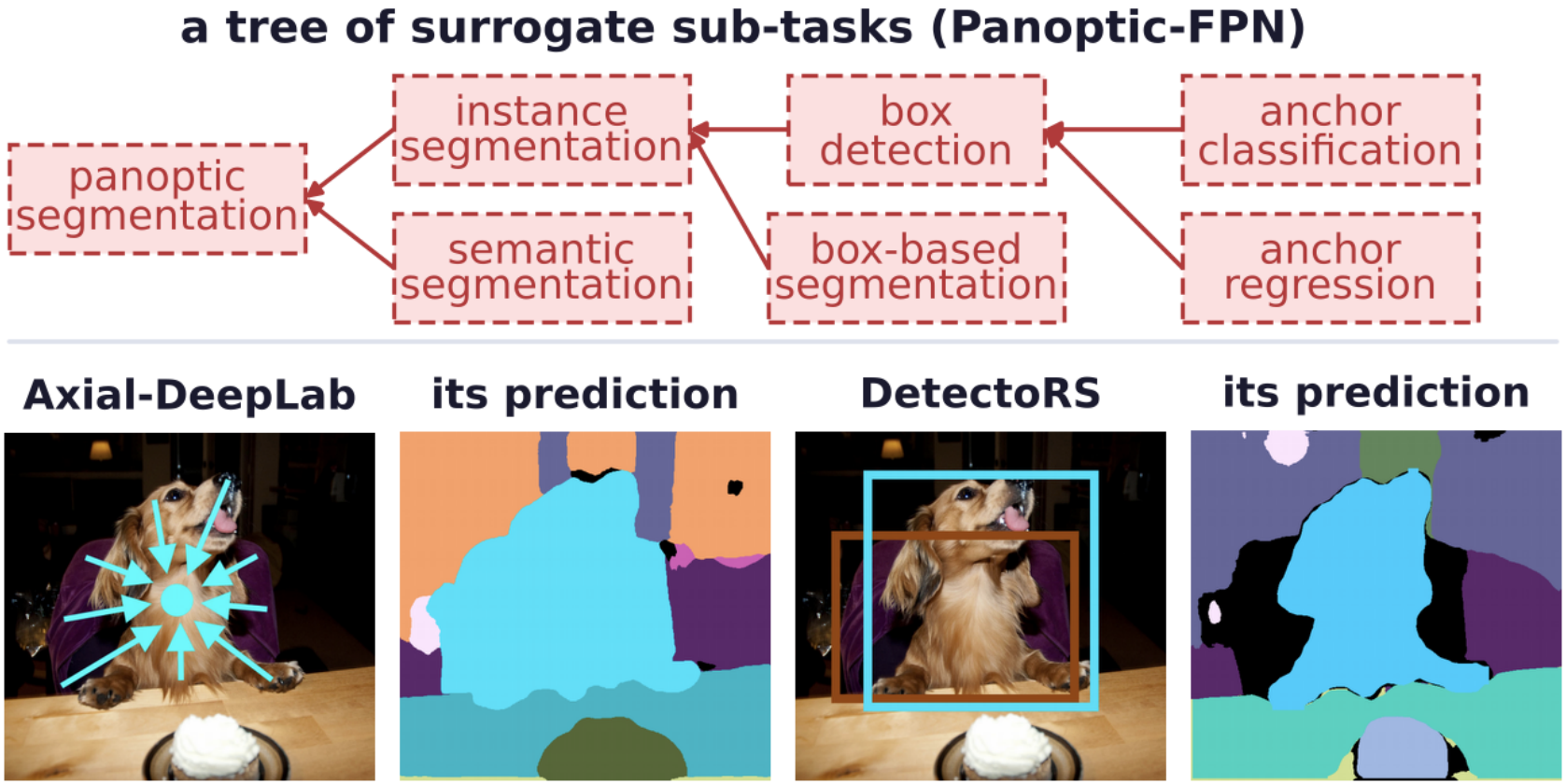


Figure 14: The tree of surrogate sub-tasks of Panoptic-FPN (after Wang et al., 2021, Fig. 1, HW slide 32, redrawn; each arrow runs from a sub-task to the task it serves), and two failures (HW slide 33, from Wang et al., 2021, Fig. 2). Axial-DeepLab's predicted center offsets (cyan arrows) meet in one center, and its prediction merges the dog and the chair. DetectoRS's boxes (the dog's in cyan, the chair's in brown) and its prediction, which misses the chair (black: unlabeled).

MaX-DeepLab predicts N masks and N classes end to end

MaX-DeepLab (Wang et al., 2021) predicts the panoptic segmentation directly, without boxes, anchors or centers. Its mask transformer updates $N = 128$ latent vectors c_g, the memory, by attention with the pixel features; each c_g gives one mask and one class:

$$\hat{m}_g(p) = \frac{\exp(x_p^T c_g)}{\sum_{g'=1}^{N} \exp(x_p^T c_{g'})}, \qquad \hat{p}_g(\ell), \quad \ell \in \{\text{classes}\} \cup \{\varnothing\}.$$

At each pixel the N mask probabilities sum to 1 (teal): the masks compete for the pixel. The classes ℓ are the things, the stuff and $\varnothing$, no object. At inference each mask takes its most probable class and each pixel its most probable mask; masks of class $\varnothing$ are discarded.

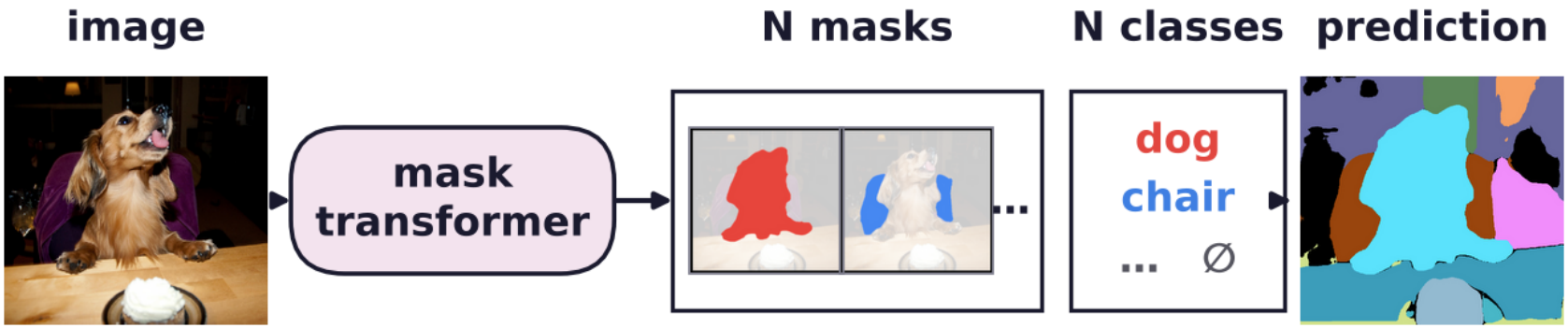


Figure 15: MaX-DeepLab end to end (HW slides 34–35, redrawn); the colors are arbitrary.

A dual-path transformer couples pixels and memory

The pixels and the memory form two paths that exchange information in every block. The pixel path is a CNN with axial attention among the pixels; the memory path holds the N vectors c_g, which store non-local information. Two attentions couple them; q, k, v carry the index of their source (q_g, k_g, v_g are projections of c_g; q_p, k_p, v_p of x_p):

$$\text{memory update:} \quad c_g \leftarrow c_g + \sum_{n \in \text{pixels} \cup \text{memory}} \text{softmax}_n(q_g^T k_n)\, v_n$$

$$\text{pixel update:} \quad x_p \leftarrow x_p + \sum_{g=1}^{N} \text{softmax}_g(q_p^T k_g)\, v_g$$

In the memory update each memory vector queries all the pixels and all the memory vectors, with one softmax over the $HW + N$ of them; it brings pixel information into the memory. In the pixel update each pixel queries the N memory vectors, with weights that sum to 1 over them (teal); this feedback lets the pixel path refine its features with information about the objects.

A dual-path transformer couples pixels and memory

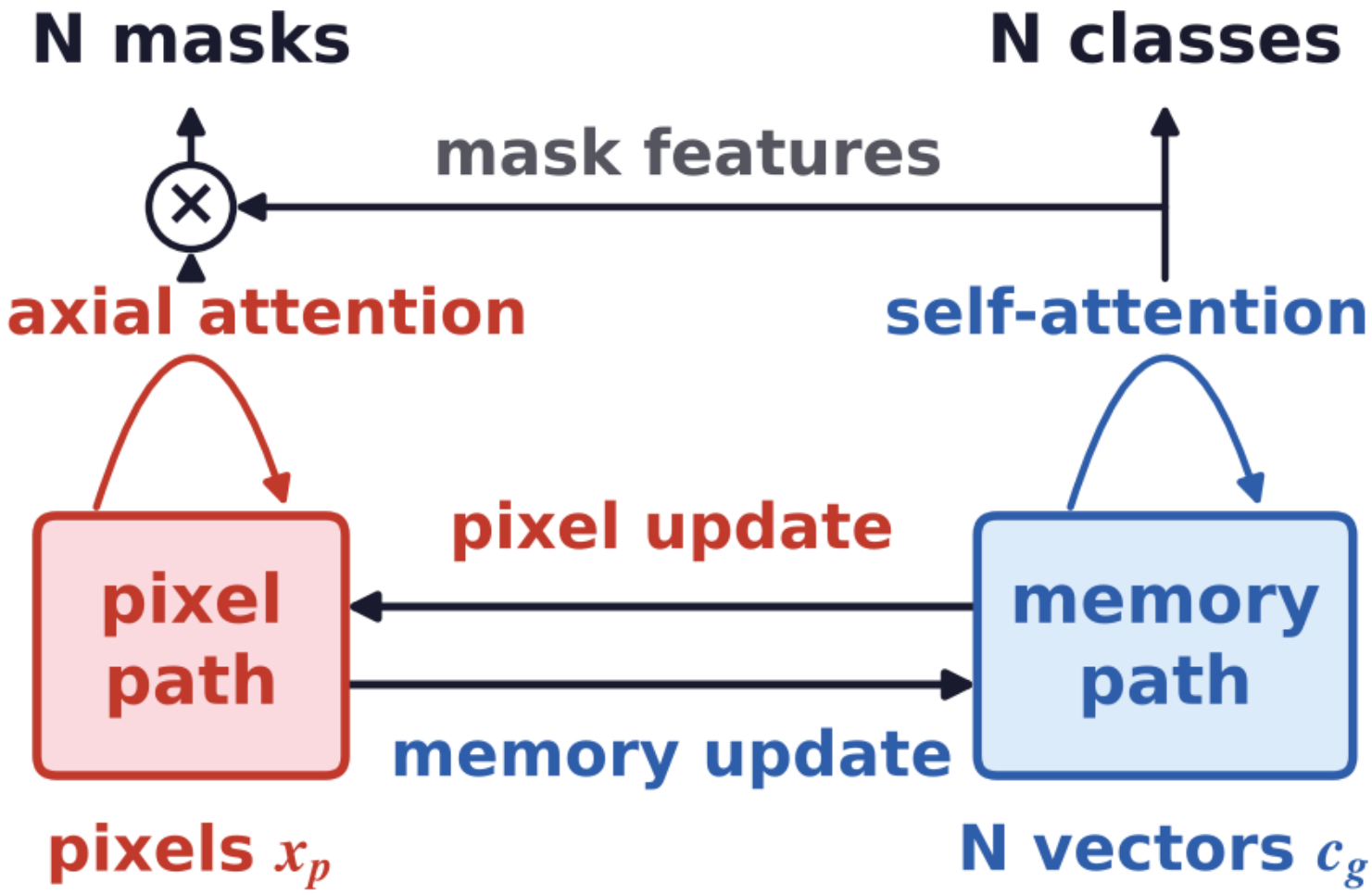


Figure 16: The dual-path transformer (after HW slide 37, redrawn): pixel path (red) and memory path (blue), each with its own attention (loops); arrows labeled by what they update. Pixels and memory give the masks ($\times$: $x_p^T c_g$), the memory the classes.

Hungarian matching pairs predicted and true masks

Training compares K true masks m_i with classes ℓ_i to $N > K$ predicted masks $\hat{m}_g$ with class probabilities $\hat{p}_g$, in no particular order.
MaX-DeepLab scores a pair by a PQ-style similarity:

$$\mathrm{sim}(i, g) = \hat{p}_g(\ell_i)\ \mathrm{Dice}(m_i, \hat{m}_g), \qquad \mathrm{Dice}(m, \hat{m}) = \frac{2 \sum_p m \hat{m}}{\sum_p m + \sum_p \hat{m}},$$

twice the overlap over the total area of the soft masks. A matching π gives each true mask i its own prediction $\pi(i)$; the best matching maximizes the total similarity,

$$\hat{\pi} = \arg\max_{\pi} \sum_{i=1}^{K} \mathrm{sim}(i, \pi(i)).$$

The Hungarian algorithm (Kuhn, 1955) solves this assignment problem in polynomial time ($O(N^3)$ in its standard modern form), instead of trying all $N!/(N-K)!$ matchings; DETR (detection transformer; Carion et al., 2020) uses it for boxes.

Hungarian matching pairs predicted and true masks

The loss raises the similarity of the matched pairs and trains the unmatched masks to predict $\varnothing$. Write $\hat{p}_i$, $\hat{m}_i$ for the prediction matched to true mask i:

$$L^+ = \sum_{i=1}^{K} \Big(\hat{p}_i(\ell_i) \left[-\,\mathrm{Dice}(m_i, \hat{m}_i) \right] + \mathrm{Dice}(m_i, \hat{m}_i) \left[-\log \hat{p}_i(\ell_i) \right] \Big),$$

$$L^- = \sum_{g \ \text{unmatched}} -\log \hat{p}_g(\varnothing),$$

combined as $0.75\,L^+ + 0.25\,L^-$. Each factor of the similarity is trained with the other as a weight held fixed (no gradient flows through it), and the class term uses $-\log \hat{p}$, the usual cross-entropy: a mask with a confident class learns to segment better, and a well-segmented mask learns its class.

Hungarian matching pairs predicted and true masks

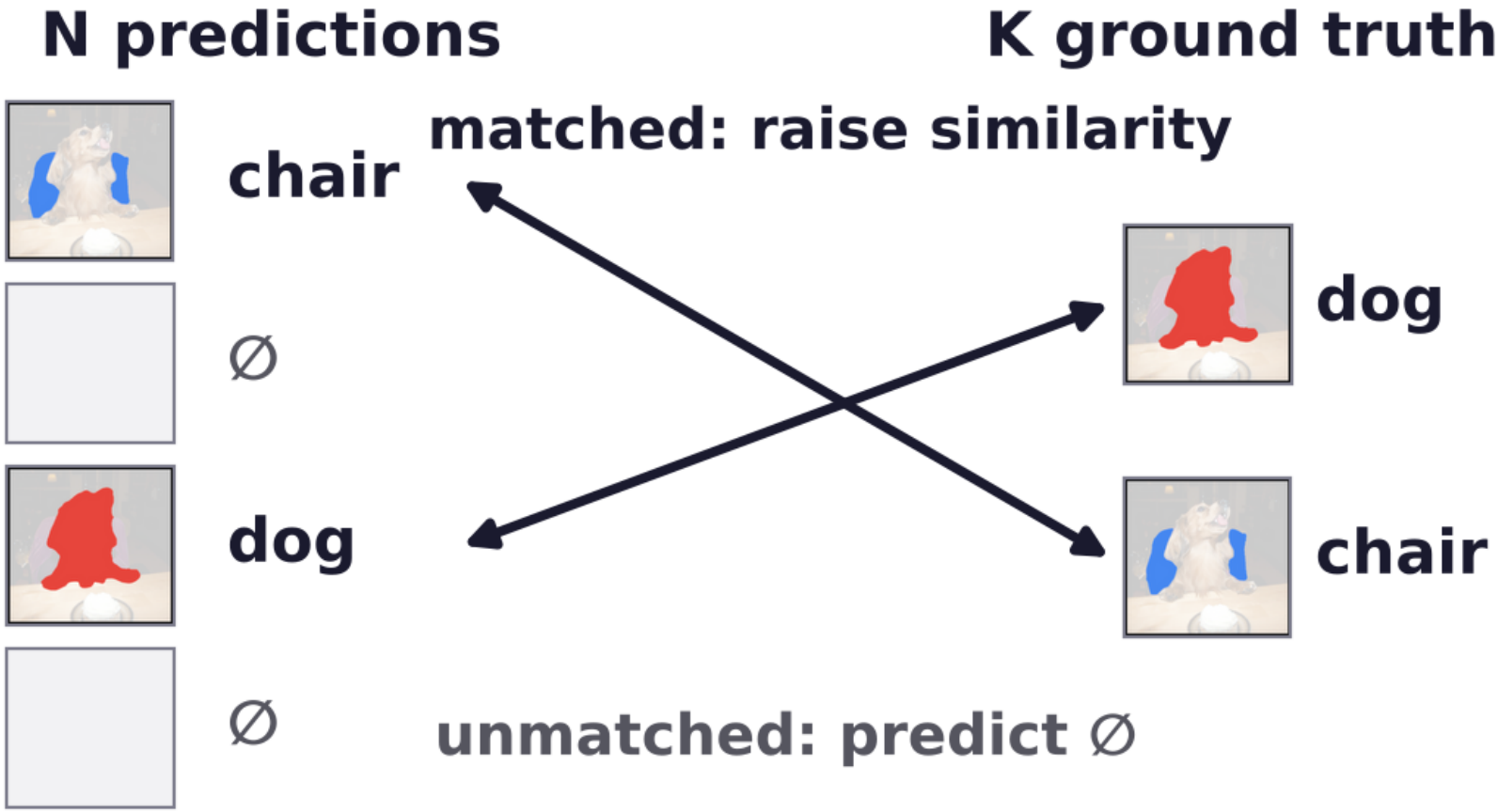


Figure 17: Matching for the dog image (after HW slide 39, redrawn; illustrative). Two true masks are matched to two of the predictions; the unmatched predictions are trained toward ∅.

MaX-DeepLab outperforms box-based methods on COCO

COCO test-dev is the held-out test set, scored by the COCO server; PQ is given for all classes, things (Th) and stuff (St); S and L are small and large models. UPSNet, DetectoRS and Panoptic-DeepLab are earlier panoptic methods. TTA, test-time augmentation, averages the predictions over flipped and rescaled copies of the image.

	COCO test-dev	TTA	PQ	PQ Th	PQ St
box-based	Panoptic-FPN		40.9	48.3	29.7
	DETR		46.0	–	–
	UPSNet	✓	46.6	53.2	36.7
	DetectoRS	✓	49.6	57.8	37.1
box-free	Panoptic-DeepLab	✓	41.4	45.1	35.9
	Axial-DeepLab-L		43.6	48.9	35.6
	Axial-DeepLab-L	✓	44.2	49.2	36.8
	MaX-DeepLab-S		49.0	54.0	41.6
	MaX-DeepLab-L		**51.3**	57.2	42.4

Box-free MaX-DeepLab-L has the highest PQ, above every box-based method, without test-time augmentation; DetectoRS with augmentation keeps a higher PQ on things (57.8 against 57.2).

MaX-DeepLab outperforms box-based methods on COCO

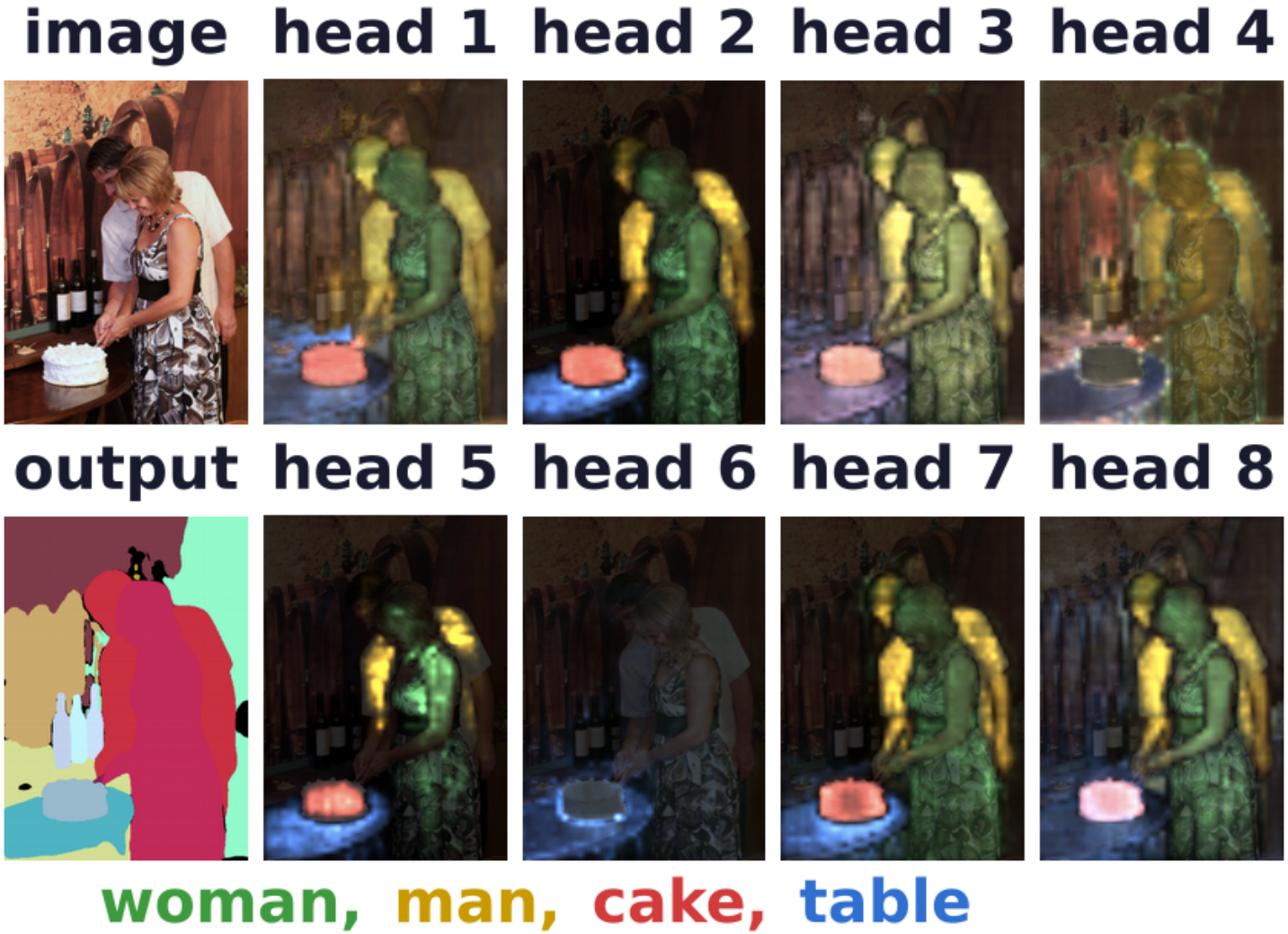


Figure 18: Memory-to-pixel attention of eight heads for the masks of a woman (green), a man (yellow), a cake (red) and a table (blue); output: the prediction (HW slide 45). Heads 2, 5, 7, 8 attend mainly to the masks; heads 1, 3, 4 gather wider context.

Mask transformers update object queries by cross-attention

DETR, MaX-DeepLab and MaskFormer (Cheng et al., 2021) share one pipeline. A backbone extracts image features; a transformer encoder enhances them by self-attention; learnable object queries gather pixel features through a transformer decoder (cross-attention) into box or mask embeddings; and Hungarian matching assigns the predictions to the ground truth for training. The latent vectors c_g are now called object queries (their projections q_g are the queries of the attention). One cross-attention update of query g takes three steps:

1. affinity logits: $\ell_{gp} = q_g^T k_p = (W^Q c_g)^T (W^K x_p)$
2. softmax over the pixels: $a_{gp} = \dfrac{\exp(\ell_{gp})}{\sum_{p'=1}^{HW} \exp(\ell_{gp'})}$
3. values, residual update: $c_g \leftarrow c_g + \sum_p a_{gp} W^V x_p$

The logit ℓ_{gp} is unrelated to the class ℓ_i. The softmax runs over the HW pixels (orange): each query's weights over the image sum to 1. The update is a weighted average of the pixel values, added to the query.

Mask transformers update object queries by cross-attention

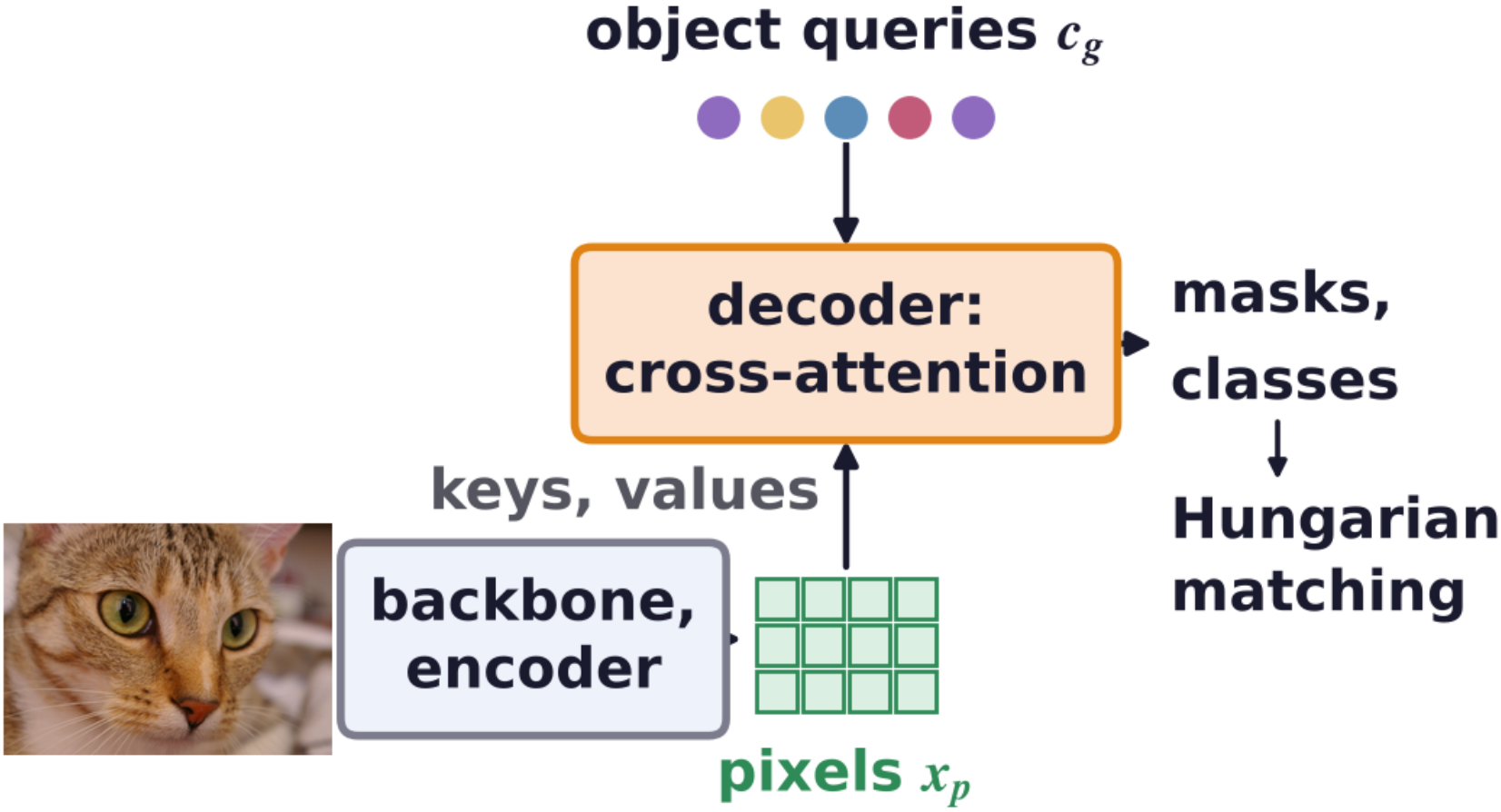


Figure 19: The common pipeline of mask transformers (after the DETR, MaskFormer and MaX-DeepLab figures on TR slide 47, drawn as one): backbone, transformer encoder, transformer decoder in which the object queries attend to the pixel features, and Hungarian matching to the ground truth.

Mask prediction and query update normalize differently

The object queries play two roles in a mask transformer, and the two roles normalize similar affinities along different axes:

$$\hat{m}_g(p) = \frac{\exp(x_p^T c_g)}{\sum_{g'=1}^{N} \exp(x_p^T c_{g'})}, \qquad a_{gp} = \frac{\exp(q_g^T k_p)}{\sum_{p'=1}^{HW} \exp(q_g^T k_{p'})}.$$

In the mask prediction $\hat{m}_g(p)$, at each pixel the probabilities over the N queries sum to 1 (teal): the queries compete for the pixel. In the query update a_{gp}, for each query the weights over the HW pixels sum to 1 (orange): the pixels compete for the query. The affinities are similar (final features and queries, or their projections) but normalized differently, so a query can gather its update from pixels that its mask does not claim. CMT-DeepLab calls this an inconsistency.

The update is also sparse: with a softmax over thousands of pixels, the weights tend to concentrate on a few locations, so each query is updated from a few pixels only (CMT-DeepLab). MaX-DeepLab's attention maps for a person query and a tree query show a few red spots.

Mask prediction and query update normalize differently

In decoders without a pixel update, such as those of DETR and MaskFormer, the pixels also communicate with the object queries only once, in the final mask prediction. CMT-DeepLab addresses both problems by reading cross-attention as clustering.



Figure 20: Attention maps of MaX-DeepLab's person query and tree query (TR slide 64): the weights of each query over the pixels (blue: small, red: large) concentrate on a few spots.

Cross-attention as clustering: each pixel chooses an object

CMT-DeepLab (Yu et al., 2022a) maps cross-attention onto clustering: the object queries are cluster centers, the attention map is a clustering assignment, and the update of the queries is the update of the centers. The analogy exposes the problem. In translation each object query corresponds to a word of the target language, and the softmax over the source words assigns it to its most affiliated source word, as it should. In panoptic segmentation the same softmax, now over the pixels, assigns each object to its most affiliated pixels: read as clustering, each object looks for its few best pixels. We expect the opposite, many pixels in one object. The softmax over the spatial dimension inverts the assignment between objects and pixels.

Normalized over the objects instead, each pixel chooses its most affiliated object, and all the pixels assigned to an object serve as its update. This is the assignment step of k-means clustering (Lecture 8).

Cross-attention as clustering: each pixel chooses an object

The figure computes both normalizations for eight pixels of the photograph, four from each of two color clusters: k-means with $k = 2$ on the Lab colors gives the centers c_1, c_2, drawn in their mean colors, and the logits are $4\, x_p^T c_g$ on standardized colors. Over the pixels (orange) each center's weights sum to 1 over the eight pixels and fall unevenly on the four of its color (0.18 to 0.36). Over the objects (teal) each pixel gives 0.94 or more to the center of its color.

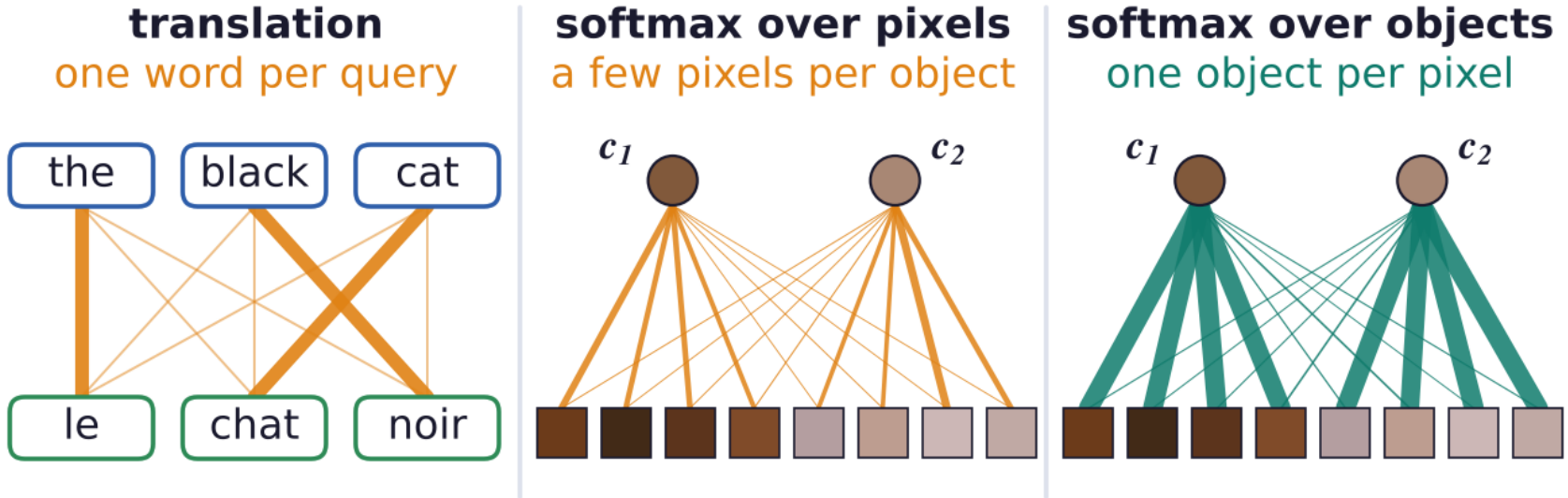


Figure 21: Line width is the weight. Translation: English words (blue) query French words (green), with illustrative weights; segmentation: centers c_1, c_2 and eight pixels, with the weights normalized over the pixels or over the objects.

CMT-DeepLab assigns pixels to clusters, then updates both

CMT-DeepLab adds two steps to each decoder layer. The assignment step assigns each pixel softly to the N clusters, like a mask prediction:

$$Z_{pg} = \frac{\exp(s_{pg} + \tilde{k}_p^T \tilde{q}_g)}{\sum_{g'} \exp(s_{pg'} + \tilde{k}_p^T \tilde{q}_{g'})},$$

with $\tilde{k}_p$, $\tilde{q}_g$ second projections of the pixels and centers, and s_{pg} the logits of the previous layer; Z is trained with the true masks. The update step adds Z to the cross-attention weights a_{gp} (orange) and updates the pixels too:

$$\text{cluster update:} \quad c_g \leftarrow c_g + \sum_p \left(a_{gp} + Z_{pg}\right) v_p$$

$$\text{pixel update:} \quad x_p \leftarrow x_p + \sum_g Z_{pg}\, v_g^c$$

with v_g^c a value projected from c_g (v_g in the dual-path notation). Pixels and centers are refined together, as in k-means.

CMT-DeepLab assigns pixels to clusters, then updates both

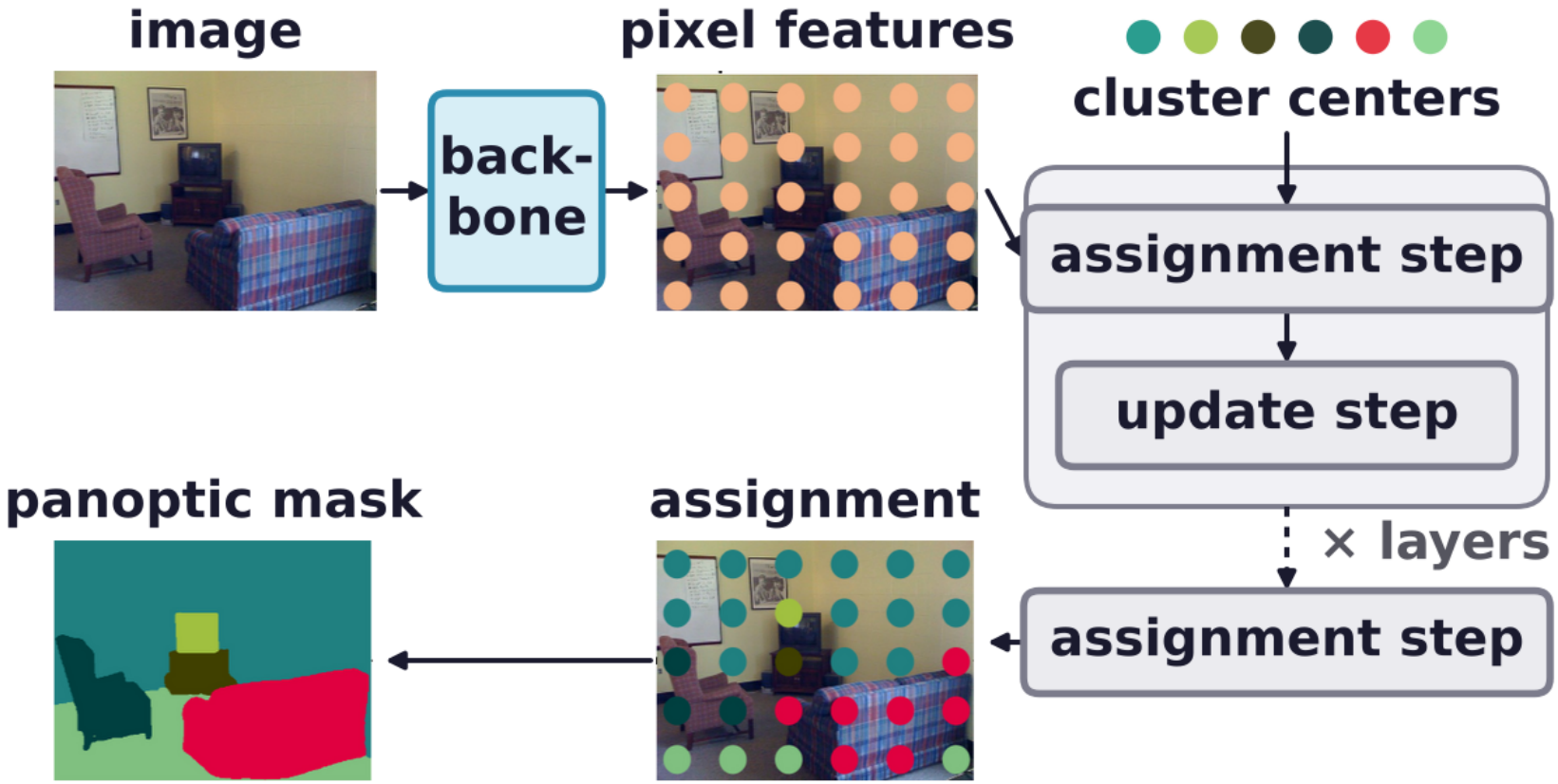


Figure 22: CMT-DeepLab (after TR slide 74, redrawn with its four pictures): the backbone gives the pixel features (orange dots); the pixels and the cluster centers (colored dots) pass through the assignment step and the update step in each decoder layer; the final assignment (each pixel colored by its center) gives the panoptic mask.

Clustering gives denser attention and better panoptic quality

CMT-DeepLab also adds location-sensitive clustering (each cluster predicts reference coordinates, and the pixel features receive their coordinates) and a pixel-wise contrastive loss. The table compares PQ for the largest models of each paper on COCO, and for the small models on Cityscapes val.

PQ	COCO val, largest	COCO test-dev, largest	Cityscapes val, S
MaX-DeepLab	51.1	51.3	61.7
CMT-DeepLab	**55.3**	**55.7**	**64.6**

The largest CMT-DeepLab, with an axial ResNet-104 backbone, is compared with MaX-DeepLab-L: +4.2 PQ on COCO val and +4.4 on test-dev. The small models differ by a similar amount on COCO (53.0 against 48.4 on val) and by +2.9 on Cityscapes. The gains are those of the whole model, which also includes the location-sensitive clustering and the contrastive loss.

Clustering gives denser attention and better panoptic quality

In the last decoder layer, with the heads averaged, the attention of each predicted mask is sparse in MaX-DeepLab and covers the object in CMT-DeepLab.



Figure 23: Last-layer attention for five predicted masks (three people, two sheep) and the panoptic prediction, in MaX-DeepLab and in CMT-DeepLab (TR slide 81, from Yu et al., 2022a, Fig. 1).

k-means alternates an assignment step and an update step

k-means clustering (Lecture 8) groups the pixel features x_p into k clusters (k is unrelated to the keys k_p) with centers c_g by alternating two steps:

$$V_{pg} = \begin{cases} 1 & \text{if } g = \arg\max_{g'} x_p^T c_{g'} \\ 0 & \text{otherwise,} \end{cases} \qquad c_g = \frac{\sum_p V_{pg}\, x_p}{\sum_p V_{pg}}.$$

V_{pg} is a binary assignment variable with $\sum_g V_{pg} = 1$: each pixel joins one cluster. Ordinary k-means assigns each pixel to the nearest center. Since $|x_p - c_g|^2 = |x_p|^2 - 2x_p^T c_g + |c_g|^2$, the nearest center is the one with the largest dot product when all the centers have the same length. The update makes each center the mean of its pixels. With the nearest-center assignment no step raises the sum of squared distances, so the algorithm stops after finitely many steps, at a local minimum.

k-means alternates an assignment step and an update step

The figure runs k-means on the colors of the photograph. After the first assignments one mid-brown cluster covers most of the fur; at convergence the four clusters are the dark stripes and pupils, two shades of fur, and the pale background and white fur.

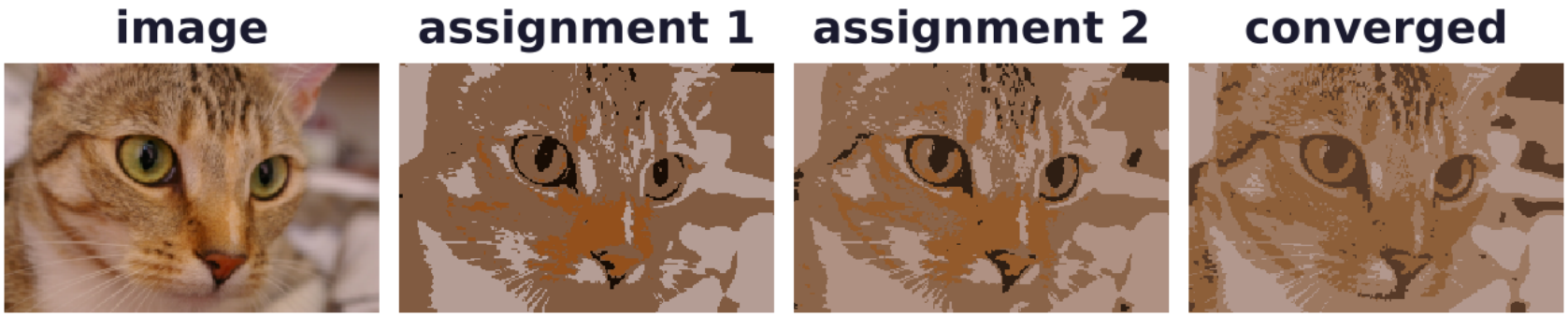


Figure 24: k-means with $k = 4$ on the Lab colors of all 33,900 pixels of a 150×226 copy of the photograph (nearest center; initial centers spread apart by k-means++), each pixel painted in the mean color of its cluster: after assignments 1 and 2, and at convergence (49 iterations).

The constraint on the assignment decides the normalization

Soft assignments come from a Gibbs distribution over the binary assignments V, with an energy that is low when pixels go to similar centers:

$$E(V, c) = -\sum_{p,g} V_{pg}\, x_p^T c_g, \qquad P(V) \propto e^{-E(V,c)}.$$

The expected value $\langle V_{pg} \rangle$, the probability that pixel p is assigned to cluster g, depends on the constraint on V.

Each pixel joins one cluster: $\sum_g V_{pg} = 1$ for every p. With $g(p)$ the cluster of pixel p, $P(V) \propto \prod_p \exp(x_p^T c_{g(p)})$ is a product of independent factors, so each pixel chooses independently:

$$\langle V_{pg} \rangle = \frac{\exp(x_p^T c_g)}{\sum_{g'} \exp(x_p^T c_{g'})},$$

a normalization over the clusters (teal): soft k-means, and the form of the mask prediction.

Each cluster taking one pixel gives the cross-attention update

Each cluster takes one pixel: $\sum_p V_{pg} = 1$ for every g. Now V is a choice of one pixel $p(g)$ for each cluster, $P(V) \propto \prod_g \exp(x_{p(g)}^T c_g)$ factorizes over the clusters, and

$$\langle V_{pg} \rangle = \frac{\exp(x_p^T c_g)}{\sum_{p'} \exp(x_{p'}^T c_g)},$$

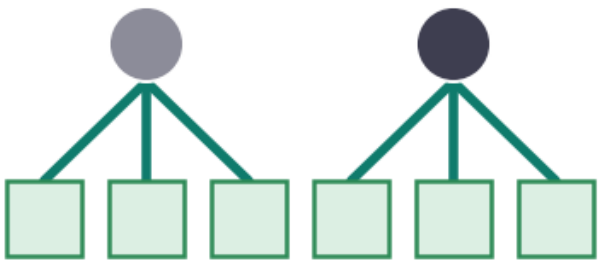
a normalization over the pixels (orange). The pixel chosen by cluster g has the expected feature

$$\sum_p \langle V_{pg} \rangle \, x_p = \sum_p \frac{\exp(x_p^T c_g)}{\sum_{p'} \exp(x_{p'}^T c_g)} \, x_p,$$

which is the cross-attention update with W^Q, W^K and W^V the identity and without the residual. Cross-attention solves a correspondence problem: each cluster, or object query, looks for one matching pixel, as a target word looks for its source word. Two clusters may choose the same pixel.

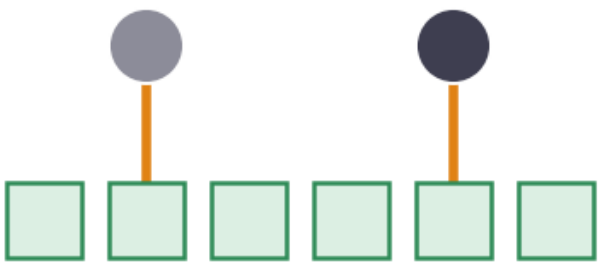
The constraint on the assignment decides the normalization

Alternating the soft assignment with an update of the centers gives an iterative algorithm in either case. Normalized over the clusters it clusters the pixels; normalized over the pixels it finds, for each center, a corresponding pixel.



each pixel joins one cluster

a clustering (k-means, masks)



each cluster takes one pixel

correspondence (cross-attention)

Figure 25: The two constraints on six pixels (green squares) and two clusters (gray and dark dots); each line is one $V_{pg} = 1$.

Normalizing over the clusters is Gaussian clustering

For a Gaussian mixture with equal priors and covariance $\sigma^2 I$ (Lecture 8), the posterior probability that x_p came from cluster g is

$$S_{pg} = \frac{e^{-|x_p - c_g|^2/2\sigma^2}}{\sum_{g'} e^{-|x_p - c_{g'}|^2/2\sigma^2}}.$$

Expand the square: $-|x_p - c_g|^2/2\sigma^2 = (x_p^T c_g - |x_p|^2/2 - |c_g|^2/2)/\sigma^2$. $|x_p|^2$ is the same for every g and cancels in the ratio; $|c_g|^2$ cancels too when all the centers have the same length. Then

$$S_{pg} = \frac{e^{x_p^T c_g/\sigma^2}}{\sum_{g'} e^{x_p^T c_{g'}/\sigma^2}},$$

the softmax over the clusters of the dot products, at temperature σ^2 (unequal lengths add $-|c_g|^2/2\sigma^2$ to the logits). As $\sigma \to 0$ the largest logit takes all the weight: S_{pg} tends to 1 for the nearest center and to 0 for the others, and the soft assignment becomes the hard assignment of k-means. The argmax over the clusters is the limit of Gaussian clustering.

Normalizing over the clusters is Gaussian clustering

The figure normalizes Gaussian weights both ways: k-means with $k = 3$ gives the color centers c_1–c_3, and the logits are $-|x_p - c_g|^2/2\sigma^2$, $\sigma^2 = 1/4$, on standardized Lab colors. Over the pixels each map is a distribution over the image; over the clusters the three maps sum to 1 at each pixel, the brightest being the pixel's k-means cluster.



Figure 26: Each center's weights normalized over the pixels (each map scaled to its own maximum), and normalized over the three clusters. Bright: large.

Cross-attention and k-means clustering differ in three ways

Both perform four steps: affinity logits, weights, values, update. Linear projections: cross-attention learns W^Q, W^K, W^V; k-means has none. Residual update: cross-attention adds to the old c_g; k-means replaces it by the new mean. Normalization: a softmax over the pixels, or an argmax over the clusters; the kMaX authors consider this difference the most significant.

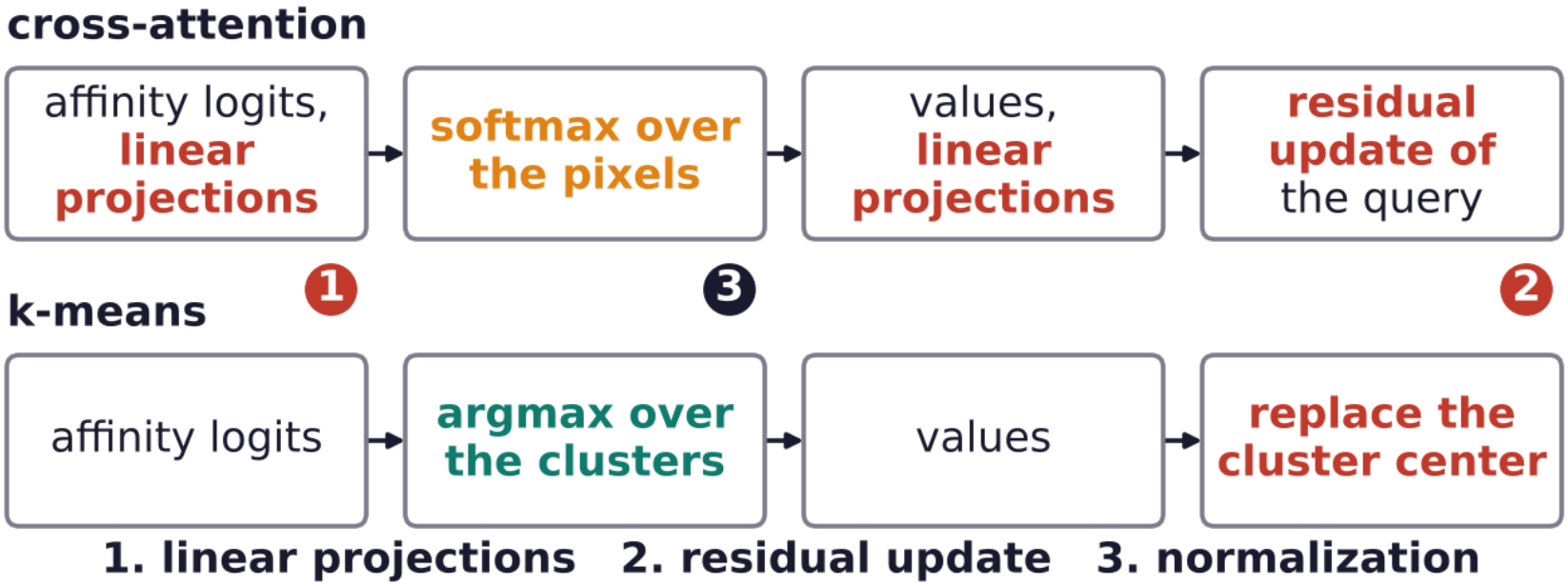


Figure 27: Cross-attention and k-means (after TR slides 94–98, redrawn); the badges mark the three differences.

k-means cross-attention takes an argmax over the clusters

kMaX-DeepLab (Yu et al., 2022b) makes the third change only:

$$\text{cross-attention:} \quad c_g \leftarrow c_g + \sum_p \frac{\exp(q_g^T k_p)}{\sum_{p'} \exp(q_g^T k_{p'})} v_p$$

$$\text{k-means cross-attention:} \quad c_g \leftarrow c_g + \sum_p V_{pg}\, v_p,$$

$$V_{pg} = 1 \text{ if } g = \arg\max_{g'} q_{g'}^T k_p, \text{ else } 0$$

The values of the pixels of cluster g are summed into c_g; k-means would divide by their number. The argmax has no gradient; of the remedies tried, deep supervision worked best: the affinities $q_g^T k_p$ of every kMaX decoder also serve as mask logits and receive the mask loss. This works because the attention map, normalized over the clusters like the mask prediction, is a segmentation that can be supervised. Added to ordinary cross-attention, deep supervision gives -1.2 PQ with ResNet-50 and $+0.1$ with MaX-S (a ResNet-50 with axial attention): the gain needs the new normalization.

k-means cross-attention takes an argmax over the clusters

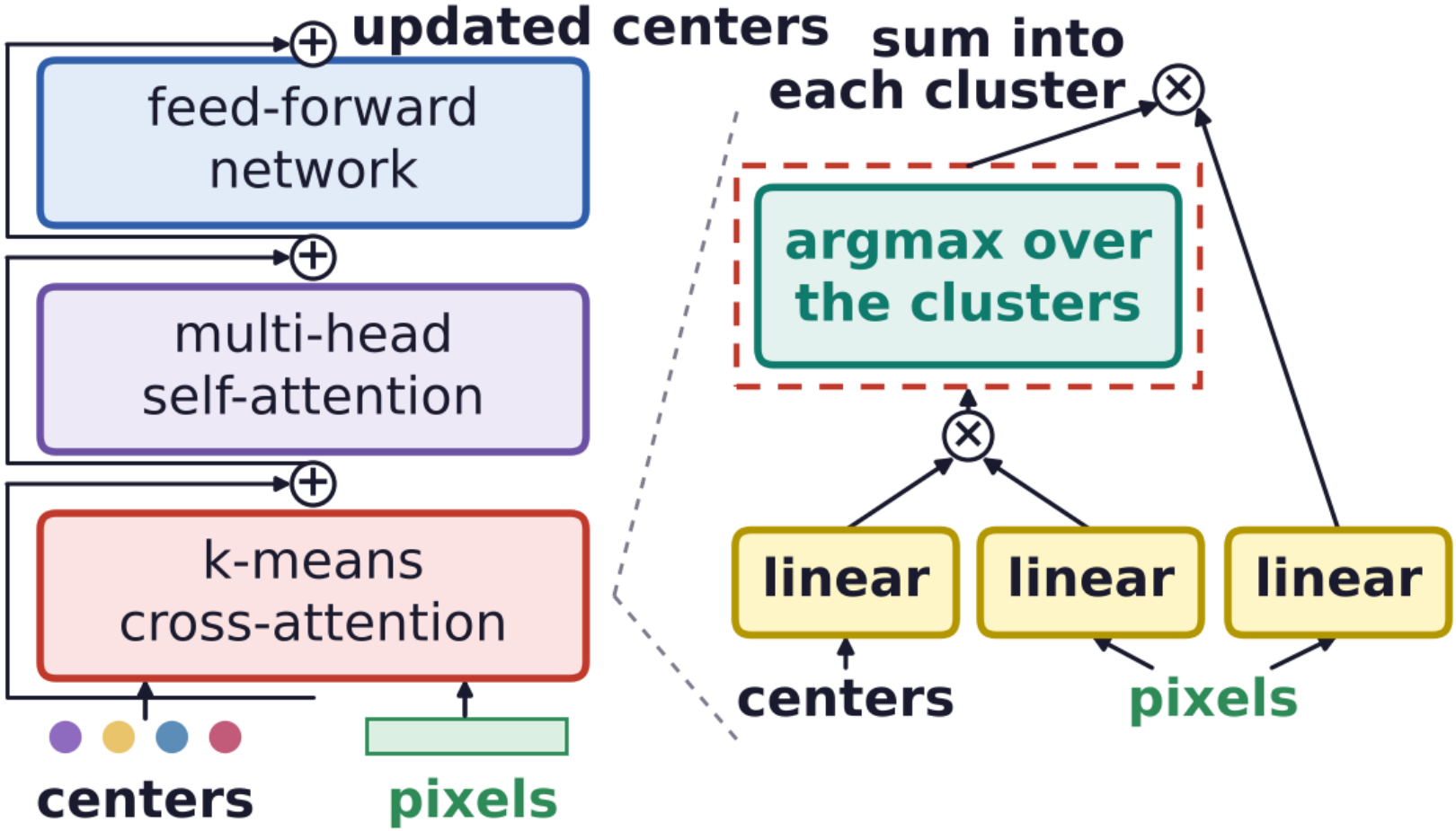


Figure 28: The kMaX decoder (after TR slide 100, redrawn): k-means cross-attention, multi-head self-attention among the centers and a feed-forward network, each with a residual connection (+). Inside the k-means cross-attention: linear projections of the centers and the pixels, their products, the argmax over the clusters (in the red box, the one change) and the sum of the pixel values into each cluster.

The simple change improves PQ at negligible extra cost

The ablation trains on COCO for 100k iterations, shorter than the final models, and changes only the module in which pixels and clusters interact. Params in millions (M), FLOPs (floating-point operations per image) in billions (G); MaX-S is a ResNet-50 with axial attention in its third and fourth stages.

	ResNet-50			MaX-S		
pixel-cluster interaction	params	FLOPs	PQ	params	FLOPs	PQ
cross-attention	56M	165G	47.5	73M	237G	52.0
dual-path cross-attention	58M	175G	48.0	75M	247G	52.3
k-means cross-attention	57M	168G	52.7	74M	240G	56.1
dual-path k-means cross-attention	59M	176G	53.0	76M	248G	56.2

With ResNet-50 one change raises PQ from 47.5 to 52.7 (+5.2), for 1M more parameters and 3G more FLOPs; with MaX-S from 52.0 to 56.1. The dual-path versions, in which the pixels also attend to the clusters, add little (0.5 PQ or less), and kMaX does not use them. The paper has no ablation of the argmax against a softmax over the clusters.

kMaX-DeepLab updates cluster centers at three resolutions

The pixel encoder is any backbone; the enhanced pixel decoder refines its features with axial-attention blocks and upsamples them to 1/4 resolution. Six kMaX decoders, two at each of 1/32, 1/16 and 1/8 resolution, update $N = 128$ cluster centers (256 on Cityscapes), which start as learned embeddings. The final centers give the classes and the masks, a softmax over the clusters of $x_p^T c_g$.

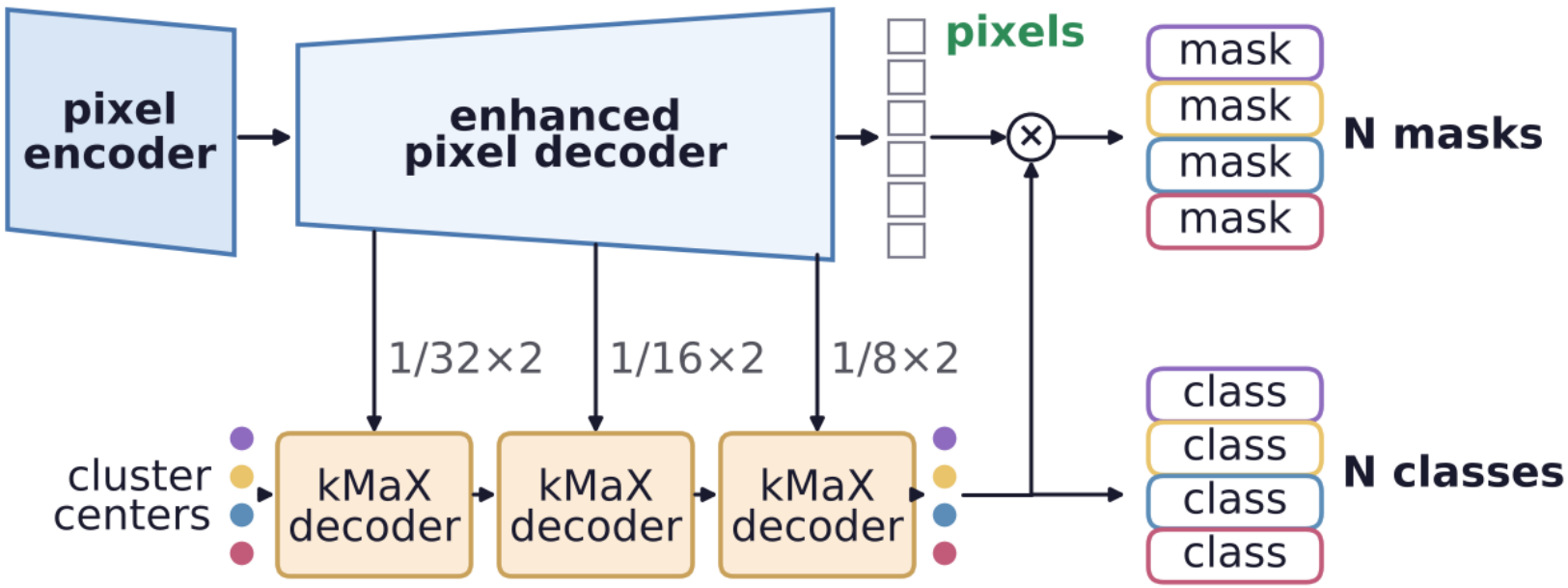


Figure 29: kMaX-DeepLab (after TR slide 103, redrawn). Each decoder box stands for two kMaX decoders; dots: cluster centers.

kMaX-DeepLab improves PQ on COCO and Cityscapes

Each method appears with its strongest backbone, named as in the papers (L: large, XL: extra large); FPS: frames per second.

COCO val	backbone	params	FLOPs	FPS	PQ
MaX-DeepLab	MaX-L	451M	3692G	–	51.1
MaskFormer	Swin-L	212M	792G	5.2	52.7
K-Net	Swin-L	–	–	–	54.6
CMT-DeepLab	Axial-R104-RFN	270M	1114G	3.2	55.3
kMaX-DeepLab	ConvNeXt-L	232M	**749G**	**6.6**	**58.0**

Cityscapes val	backbone	params	FLOPs	FPS	PQ	AP	mIoU
Panoptic-DeepLab	Xception-71	47M	548G	5.7	63.0	35.3	80.5
Axial-DeepLab	Axial-ResNet-XL	173M	2447G	–	64.4	36.7	80.6
SWideRNet	(1,1,4.5)	536M	10365G	1.0	66.4	40.1	82.2
kMaX-DeepLab	ConvNeXt-L	232M	1673G	3.1	**68.4**	**44.0**	**83.5**

On COCO kMaX (here with 256 centers; 57.9 PQ with 128) beats the earlier mask transformers at fewer operations and a higher frame rate. On Cityscapes one model gives the best PQ, AP and mIoU; Panoptic-DeepLab runs faster.

Cluster assignments are refined from decoder to decoder

The figure shows the argmax assignment after each of the six kMaX decoders, then the panoptic prediction and the ground-truth label, for two images. The image is first divided into regions, which are gradually merged and refined into the final clusters.

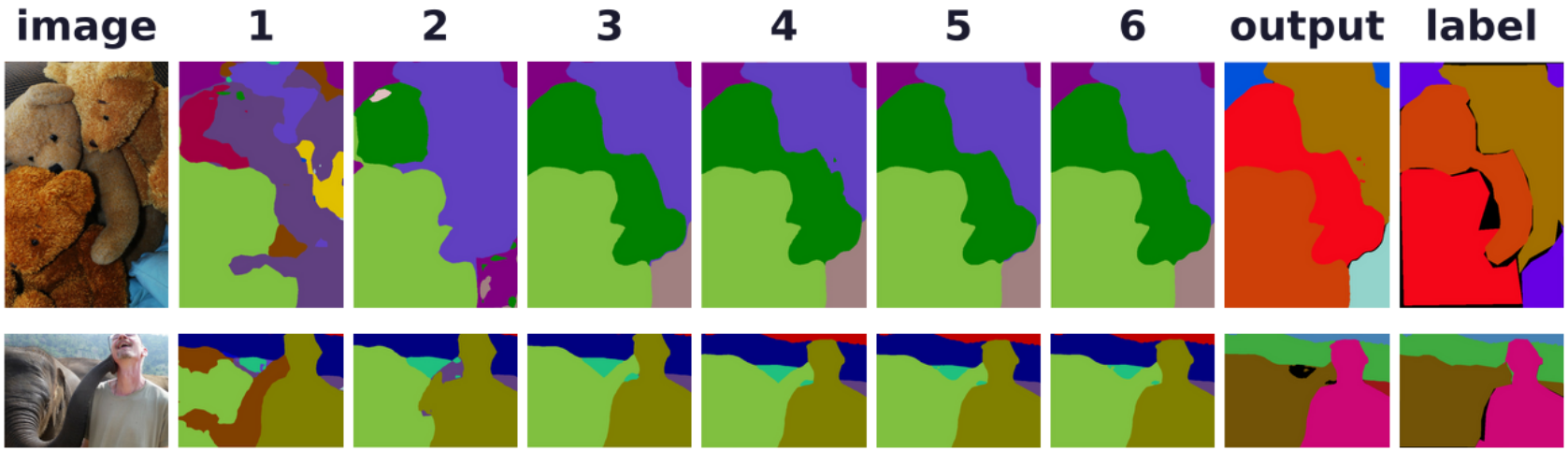


Figure 30: Cluster assignments of kMaX-DeepLab (TR slides 106–107): the image, the assignment after decoders 1 to 6, the output (the panoptic prediction, in its own colors) and the label (black: unlabeled).

Summary

- Self-attention averages values with query-key weights normalized over the keys; blocks add MLPs.
- Vision needs position, locality and scale: positional encoding, axial attention, shifted windows, hybrids.
- Cross-attention: queries from one set attend to another; in translation and stereo it finds matches.
- Axial-DeepLab: position-sensitive attention along height and width is long range at $O(HWm)$ cost.
- MaX-DeepLab: a mask transformer predicts N masks and classes end to end, with Hungarian matching.
- CMT-DeepLab: cross-attention read as clustering, in which each pixel should choose an object.
- kMaX-DeepLab: k-means cross-attention takes an argmax over the clusters; one change, much better PQ.
- The normalization decides the task: over pixels it finds correspondences, over clusters it clusters.

References


A. L. Yuille. Transformers. Course slides (TR), Johns Hopkins University, 2025.


H. Wang. Axial-DeepLab and MaX-DeepLab. Slides (HW), edited by A. L. Yuille.


A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin. Attention is all you need. *NIPS*, 5998–6008, 2017.


J. L. Ba, J. R. Kiros and G. E. Hinton. Layer normalization. arXiv:1607.06450, 2016.


W. Guo, Z. Li, Y. Yang, Z. Wang, R. H. Taylor, M. Unberath, A. Yuille and Y. Li. Context-enhanced stereo transformer. *ECCV*, 2022.


A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner et al. An image is worth 16x16 words: transformers for image recognition at scale. *ICLR*, 2021.


K. He, X. Zhang, S. Ren and J. Sun. Deep residual learning for image recognition. *CVPR*, 770–778, 2016.

References


Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin and B. Guo. Swin transformer: hierarchical vision transformer using shifted windows. *ICCV*, 10012–10022, 2021.


C. Yang, S. Qiao, Q. Yu, X. Yuan, Y. Zhu, A. Yuille, H. Adam and L.-C. Chen. MOAT: alternating mobile convolution and attention brings strong vision models. *ICLR*, 2023.


A. Kirillov, K. He, R. Girshick, C. Rother and P. Dollár. Panoptic segmentation. *CVPR*, 9404–9413, 2019.


T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár and C. L. Zitnick. Microsoft COCO: common objects in context. *ECCV*, LNCS 8693, 740–755, 2014.


M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth and B. Schiele. The Cityscapes dataset for semantic urban scene understanding. *CVPR*, 3213–3223, 2016.


P. Ramachandran, N. Parmar, A. Vaswani, I. Bello, A. Levskaya and J. Shlens. Stand-alone self-attention in vision models. *NeurIPS*, 68–80, 2019.


X. Wang, R. Girshick, A. Gupta and K. He. Non-local neural networks. *CVPR*, 7794–7803, 2018.

References


J. Ho, N. Kalchbrenner, D. Weissenborn and T. Salimans. Axial attention in multidimensional transformers. arXiv:1912.12180, 2019.


Z. Huang, X. Wang, L. Huang, C. Huang, Y. Wei and W. Liu. CCNet: criss-cross attention for semantic segmentation. *ICCV*, 603–612, 2019.


H. Wang, Y. Zhu, B. Green, H. Adam, A. Yuille and L.-C. Chen. Axial-DeepLab: stand-alone axial-attention for panoptic segmentation. *ECCV*, LNCS 12349, 108–126, 2020.


P. Shaw, J. Uszkoreit and A. Vaswani. Self-attention with relative position representations. *NAACL-HLT*, 464–468, 2018.


J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596:583–589, 2021.


A. Kirillov, R. Girshick, K. He and P. Dollár. Panoptic feature pyramid networks. *CVPR*, 6399–6408, 2019.

References


H. Wang, Y. Zhu, H. Adam, A. Yuille and L.-C. Chen. MaX-DeepLab: end-to-end panoptic segmentation with mask transformers. *CVPR*, 5463–5474, 2021.


H. W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.


N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov and S. Zagoruyko. End-to-end object detection with transformers. *ECCV*, 213–229, 2020.


B. Cheng, A. Schwing and A. Kirillov. Per-pixel classification is not all you need for semantic segmentation. *NeurIPS*, 17864–17875, 2021.


Q. Yu, H. Wang, D. Kim, S. Qiao, M. Collins, Y. Zhu, H. Adam, A. Yuille and L.-C. Chen. CMT-DeepLab: clustering mask transformers for panoptic segmentation. *CVPR*, 2560–2570, 2022a.


Q. Yu, H. Wang, S. Qiao, M. Collins, Y. Zhu, H. Adam, A. Yuille and L.-C. Chen. k-means mask transformer. *ECCV*, LNCS 13689, 288–307, 2022b.